基于几何容斥与有限步定位的无线电干扰源自动定位与清除模型

摘要

如何快速定位与清除无线电干扰源是无线电频谱管理的重要问题。本文从单个源的几何定位到全域在线清除的不同场景依次建立定位域计算、鲁棒选点、全向源搜索和定向源完备覆盖的数学模型，并用演练与正式测试检验策略效果。

针对问题一，基于多次观测对定位区域多边形凸性质的保持，本文采用了**旋转卡壳算法**计算定位区域直径，其利用凸多边形的对踵点性质，仅枚举所有对踵点即可找到凸多边形直径。在计算得出的直径 D 基础上, 我们引入多边形最小覆盖圆半径 R_*, 证明并得出结论：**以定位区域直径为直径的圆一般无法覆盖定位区域，当且仅当 $R_* = \frac{D}{2}$ 时能成立**。

针对问题二, 第二检测点的两个要求相互冲突，即交会角越大，定位越准，但偏移过大又可能因未知接收半径而收不到信号。本文将连续接收约束进行**三点化简**，求得一定能接收到信号的鲁棒候选区域。为达到在能探测到前提下的最好探测效果，我们进一步在鲁棒接收域内选择最坏交会角下界最大的点。在题设 1° 测角误差下，两次示向后定位区域直径不超过 **185.19 m**。

针对问题三，本文先以**七点完备搜索**的策略保证全向源不被漏检，再用至多**四次示向观测**确保定位已发现的源，并用滚动路线把搜索、补测与清除交错安排。在此基础上，优化策略利用原定停靠点对其他有收益的频道作机会观测，不增加额外移动目的地。在同一场景的配对演练中，优化策略使平均定位清除时间由 **435.41 s 降至 331.93 s**。三次正式测试依次清除 14、10、13 个源，对应的平均定位清除时间为 346.47、361.17、386.19 s，程序运行时间均不超过 2.34 s。

针对问题四，本文用**三角覆盖**保证任意朝向下的搜索完备性，并在同为 25 个严格检测点的条件下比较正三角网格基线方案与同心环优化方案；两版沿用相同的在线定位与清除规则。相较正三角网格，最远外扩由约 794 m 降至 **63.5 m**，固定搜索路线由 24.173 km 缩至 **17.802 km**。三次正式测试依次清除 11、15、14 个源，对应的平均定位清除时间为 633.01、504.14、578.26 s，程序运行时间均不超过 3.51 s。

模型分析表明，移动占问题三虚拟总时间的约 77%–81%、问题四的 74.10%，紧凑布点与滚动路线因而作用于主要耗时环节。问题四 40 次演练的截尾均值与原均值仅差 0.51%，平均耗时未受少数极端案例主导。模型耗时对移动速度变化较为敏感。

关键字： 交会定位；双端点化简；四次观测上界；几何容斥；在线滚动规划

一、 问题重述

目标区域为半径 1800 m 的圆域，机器狗从原点出发，通过测向机对频道进行检测。每个干扰源独占一个频道；全向源在 360° 内辐射，定向源只在定向方向两侧各 90° 内辐射，有效接收半径均介于 1000–1500 m。测向机只提供方向信息，读数与真实方位角的误差不超过 1°。需要解决四个问题：

问题一：由若干检测点坐标和对应示向度构造交会定位区域，计算其直径，并判断以该直径为直径的圆能否覆盖该区域。

问题二：已知某全向源在一点处的示向度，给出第二个检测点的选择策略及候选区域。

问题三：目标区域仅有全向源，总数 10–16 个但具体未知，设计自动搜索、定位与清除策略，使全部干扰源被清除且总时间尽量短，并报告演练和三次正式测试结果。

问题四：目标区域同时有全向源和定向源，数量、定向源个数和方向均未知，其余同问题三，给出相应策略、算法与测试结果。

二、 问题分析

2.1 问题一的分析

单次示向只限制方位，多个方向区间相交后才能形成可计算的定位域。目标圆域的边界是圆弧，直接参与直径与覆盖判定会增加几何处理；因此先用外切凸多边形形成保守外包，再以增量半平面裁剪更新区域。所得凸多边形的最远点对位于顶点上，可用旋转卡壳求得；直径圆覆盖则归结为检验一组直径端点确定的圆。问题一提供后续选点所需的定位区域、直径和最小覆盖圆。

2.2 问题二的分析

第一次示向留下的是狭长扇形，第二检测点如果追求大的视线夹角，可能超出未知的实际接收半径；若只是靠近首次检测点，又难以有效缩域。为同时控制这两种风险，我们先求出能接收到所有相容源位置的候选域。关键是把扇形外圆弧对应的连续圆盘交集化为两个端点圆盘，再加入扇形顶点的接收约束。在所得鲁棒域中优化最坏交会角下界，就能得到有接收与定位双重保证的第二检测点。

2.3 问题三的分析

二十个频道的任务会随反馈改变，缩短路线之前必须保证尚未发现的源不漏检、已发现的源能在有限次示向后清除。全向源由七个兜底点完成覆盖与判空。定位方面，前两次观测只给出一个不规则小区域。取其最小覆盖圆圆心作为第三点并取得新示向后，区域再次包含于标准窄扇形，从而能复用问题二的侧向选点结构。这个递推收缩给出四次示向上界，动态任务网络与零绕行机会观测则负责降低总任务时间。

2.4 问题四的分析

定向源即使距离足够近，也可能因检测点落在背面而无信号，此时七点圆覆盖不能再作为判空证据。对任意可能源位置，须使保证接收范围内的检测点从各方向围住它；局部凸包判据和三角证书把这一连续要求变成有限校核。正三角网格方案虽满足证书，却有较多点远离目标圆域，带来额外行程。中心—双环布点在保持同样 25 点完备性的同时压缩外扩与固定路线，随后再接入第三问的多频道在线框架。

由此，四问依次解决可行域如何计算、下一观测点如何可靠接收、全向源如何有限步清除、定向源如何在较短路线中完备搜索的问题。几何保证确定可行策略的边界，滚动调度只在这一边界内优化时间。

三、 模型假设

1. 测向机与所有干扰源位于同一水平面内，忽略高度差异，问题在二维平面内讨论。
2. 干扰源的空间尺度相对于目标区域可以忽略，均视为点源；全向源向 360° 辐射，定向源向定向方向两侧各 90° 的半平面辐射，辐射强度在有效覆盖角度内只随距离衰减。
3. 示向度误差只满足有界条件，即真实方位与读数之差落在 $[-1^\circ, 1^\circ]$ 内，不额外假设误差分布；同一检测点处的测量环境不变，重复检测不会获得新的方位信息。
4. 机器狗沿直线匀速行进，速度为 $5\ \mathrm{m/s}$，忽略加减速与转向时间；目标区域内没有影响直线行进的障碍物，机器狗的活动范围不受半径 $1800\ \mathrm{m}$ 的目标圆域限制。
5. 检测接口返回的无信号、示向度和近距离信号均视为可靠。题面误差界为 1°，接口角度保留两位小数，最坏舍入误差为 0.005°；程序中取 $\delta_{\mathrm{eff}} = 1.01^\circ$，略大于二者之和，以避免离散化漏掉真实目标。

四、 符号说明

表 1 只列出全文反复出现、会影响阅读主线的符号；少量只在单个推导或算法步骤中使用的中间量，均在对应位置随文说明。

表 **1**　主要符号说明

符号	含义	单位
O	目标圆域圆心，也是机器狗初始位置	m
$\mathcal{D}$	目标圆域，本文取 $\mathcal{D}=B(O,1800)$	—
G	干扰源位置	m
S_i	第 i 次示向检测点	m
θ_i,θ_i^*	第 i 次示向读数与真实方位角	°
$\delta,\delta_{\text{eff}}$	题设测角误差界与程序采用的有效误差界	°
$B(S,r)$	以 S 为圆心、半径为 r 的闭圆盘	—
$W(S,u;\delta)$	以 S 为顶点、中心方向为 u 的示向角域	—
ρ,ρ_-,ρ_+	实际接收半径及其下、上界	m
Ω,Ω_j	定位区域；Ω_j 表示第 j 次示向后的定位区域	—
$D(A)$	集合 A 的直径	m
$C_{\text{MEC}}(A),R_{\text{MEC}}(A)$	集合 A 的最小覆盖圆圆心与半径	m
$\mathcal{C}_{\text{rec}}$	问题二中保证下一次检测能接收信号的候选区域	—
c,z_c	频道编号及频道当前状态	—
F_c,Ω_c^{p}	频道 c 的可行域及其规划用凸外包	—
$d_k,m_k,s_k,q_k^{\text{s}},q_k^{\text{f}}$	第 k 阶段的移动距离、检测次数、换频次数、清除成功次数与清除失败次数	m / 次
$T(\pi)$	在线策略 π 的虚拟任务总时间	s

五、 问题一模型的建立与求解

5.1 示向度角域与定位区域

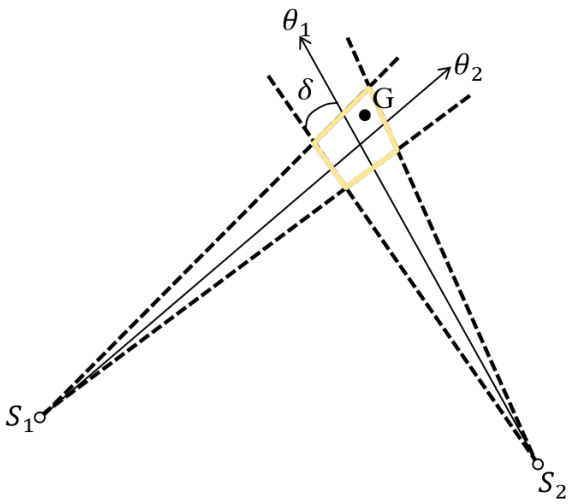


图 **1**　交会定位法的示意图

测向设备给出的示向度是一个角度读数，它与真实方位之间最多相差 $\delta=1°$[5]。设第 i 个检测点位于 $S_i=(x_i,y_i)$，在该点测得的示向度为 θ_i，则真实方向角 θ_i^* 必定落在

$[\theta_i-\delta,\,\theta_i+\delta]$ 之内。记中心方向与两条误差边界的单位向量分别为

$$u_i=\begin{bmatrix}\cos\theta_i\\ \sin\theta_i\end{bmatrix},\quad u_i^-=\begin{bmatrix}\cos(\theta_i-\delta)\\ \sin(\theta_i-\delta)\end{bmatrix},\quad u_i^+=\begin{bmatrix}\cos(\theta_i+\delta)\\ \sin(\theta_i+\delta)\end{bmatrix}, \tag{1}$$

并对二维向量 $a=(a_x,a_y)^{\mathsf T}$、$b=(b_x,b_y)^{\mathsf T}$ 记叉积 $a\times b=a_xb_y-a_yb_x$。由于角域宽度为 $2\delta=2^\circ<180^\circ$，点 X 对第 i 次观测可行，当且仅当它位于两条边界射线的内侧，即

$$u_i^-\times(X-S_i)\ge 0,\qquad u_i^+\times(X-S_i)\le 0. \tag{2}$$

两个不等式各自确定一张半平面，因此一次示向度观测给出的可行域是两张半平面之交[1]。用叉积而不直接比较角度区间有两个原因：一是它自动保留了由检测点指向干扰源的射线方向，不会把反向延长线上的点误判为可行；二是当示向度接近 0° 或 360° 时角度区间的端点会跨越一周，改用方向向量之后无须再对这种情形作特殊处理。

综合 k 次观测，记第 i 次角域为 W_i。若同时使用目标圆域 $\mathcal{D}$，仅由目标圆域与示向角域给出的相容位置集合为 $\Omega_{\text{true}}=\mathcal{D}\cap\bigcap_{i=1}^{k}W_i$，它可能带有圆弧边界。为使用多边形算法，取覆盖 $\mathcal{D}$ 的外切凸多边形 $\mathcal{P}_0$，并定义本文计算的保守定位域

$$\Omega=\mathcal{P}_0\cap\bigcap_{i=1}^{k}W_i,\qquad \Omega_{\text{true}}\subseteq\Omega. \tag{3}$$

因此后续的直径和覆盖判定均针对 Ω；它给出真实定位域的保守外包，而不把外切多边形视为目标圆域本身。Ω 在非空时为凸多边形；若 $\mathcal{P}_0$ 有 n_0 个顶点，则其顶点数至多为 n_0+2k。将其顶点按逆时针顺序记为 $P_0,P_1,\dots,P_{m-1}$。

式 (3) 的半平面更新只用检测点、示向度和误差界，不使用观测所隐含的接收距离信息；因此 Ω 可能比加入接收半径约束后的区域更大。若需得到圆域截断后的精确直径或覆盖判定，就必须显式处理圆弧边界。下面先说明增量构造与多边形直径计算。

5.2 定位区域的构造与直径计算

5.2.1 增量半平面裁剪

每获得一个新示向，就把该次读数对应的两个半平面依次加入当前凸多边形，用 Sutherland–Hodgman 裁剪更新定位域；初始先验域取覆盖目标圆域 $\mathcal{D}$ 的外切凸多边形，以避免圆域边界的圆弧处理。记半平面为 $f(X)=a^{\mathsf T}X-b\le 0$，边 AB 与边界 $f(X)=0$ 的交点参数为 $t=(b-a^{\mathsf T}A)/(a^{\mathsf T}(B-A))$；一次观测包含两次裁剪，复杂度为 $O(m)$。裁剪后删除距离过近的重复顶点和共线中间点。若顶点数 $m=0$，说明各次观测在误差范围内不相容；$m=1$ 或 $m=2$ 时，定位域分别退化为一个点或一条线段。

5.2.2 旋转卡壳与直径计算

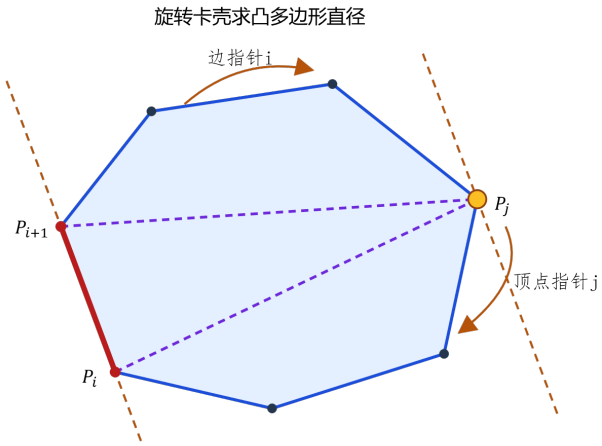


图 **2**　旋转卡壳求凸多边形直径

直径定义为区域内任意两点距离的最大值 $D(\Omega)=\max_{X,Y\in\Omega}\|X-Y\|_2$。$f(X,Y)=\|X-Y\|_2$ 在 $\Omega\times\Omega$ 上是凸函数，其最大值必在极点处取到。Ω 是凸多边形，其极点即顶点，因此最远点对必在两个顶点处取得。因此

$$D(\Omega)=\max_{0\le i<j<m}\|P_i-P_j\|_2 \tag{4}$$

旋转卡壳 [2] 利用凸多边形的对踵点结构把复杂度降到 $O(m)$：凸多边形上每条边的最远顶点随边逆时针遍历仅单向前移，用一个单调指针同步扫描所有对踵点对，以三角形面积两倍替代距离判断，全程无回溯。

5.3 直径圆覆盖的等价条件

$$\boldsymbol{\Omega\subseteq C_{AB}\Leftrightarrow R^{*}=\frac{D}{2}}$$

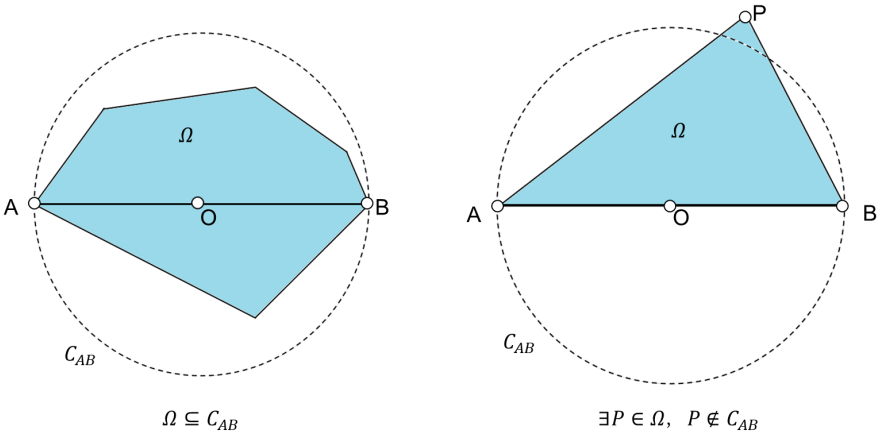


图 **3**　以 AB 为直径的圆覆盖定位区域的两种情形

题目问是否存在直径等于 D 的覆盖圆。先取旋转卡壳求得的任意一组直径端点 A, B；若半径 $D/2$ 的圆覆盖这两个端点，其圆心只能是中点 $(A+B)/2$，因此只需检验这一组端点确定的圆。

为此引入最小覆盖圆半径 [3]

$$R_* = \min_{C\in\mathbb{R}^2} \max_{X\in\Omega} \|X - C\|_2. \tag{5}$$

圆盘是凸集，一个圆覆盖 Ω 的全部顶点即覆盖整个 Ω，因此式 (5) 中的最大值只需在顶点上取。任取一对直径端点 A、B，由三角不等式

$$D = \|A - B\|_2 \le \|A - C\|_2 + \|C - B\|_2 \le 2R \tag{6}$$

可知任何覆盖圆的半径都满足 $R \ge D/2$，于是

$$R_* \ \ge \ \frac{D}{2}. \tag{7}$$

存在直径等于 D 的覆盖圆，等价于式 (7) 取到等号，即

$$\text{存在直径等于} D \text{ 的覆盖圆} \quad \Longleftrightarrow \quad R_* = \frac{D}{2}. \tag{8}$$

等号成立时式 (6) 中的三角不等式必须取等，圆心只能是该直径端点对的中点。于是对旋转卡壳给出的任意一组直径端点 A, B，判定可直接写成

$$\max_{0\le i<m} (P_i - A)^{\mathsf{T}}(P_i - B) \le 0. \tag{9}$$

该式等价于全部顶点位于以 AB 为直径的圆内；由圆盘凸性，它也等价于圆覆盖整个 Ω。即使有多组直径端点，也无需逐组尝试：若存在直径为 D 的覆盖圆，它对任意一组直径端点的圆心都必须是同一个中点。

5.4 结论

综上，对裁剪所得的凸多边形，可利用旋转卡壳法在 $O(m)$ 时间内求得直径 D 和一组直径端点，再按式 (9) 在 $O(m)$ 时间内检验其直径圆是否覆盖全部顶点。这与 $R_* = D/2$ 的条件等价。计算结果针对外切多边形形成的保守定位域；若需目标圆域截断后的精确结论，还须处理圆弧边界。

六、 问题二模型的建立与求解

6.1 第一次观测后的定位区域

设第一次检测点为 S_1，测得示向度为 θ_1。记中心示向单位向量及其法向量分别为

$$u_1 = (\cos\theta_1, \sin\theta_1)^{\mathsf{T}}, \qquad n_1 = (-\sin\theta_1, \cos\theta_1)^{\mathsf{T}}.$$

干扰源位置 G 可表示为

$$G = S_1 + t(\cos\varphi\, u_1 + \sin\varphi\, n_1), \qquad |\varphi| \le \delta,$$

其中 $\delta = 1^\circ$。实际接收半径 ρ 未知，但满足 $\rho_- = 1000\ \mathrm{m} \le \rho \le \rho_+ = 1500\ \mathrm{m}$。又因干扰源位于半径 1800 m 的圆域 $\mathcal{D}$ 内，第一次观测后的保守定位区域为

$$\Omega_1 = \mathcal{D} \cap \{S_1 + t(\cos\varphi\, u_1 + \sin\varphi\, n_1) : 0 \le t \le \rho_+,\ |\varphi| \le \delta\}. \tag{10}$$

第一次观测若已返回示向度，实际目标不会恰好位于 S_1；但保留 $t = 0$ 可得到更宽松的定位区域，便于建立不依赖目标圆域截断形状的接收保证。

6.2 选点准则

题目只告诉我们测向误差不超过 1°，并没有给出目标位置和误差的概率分布。这里我们按最坏情况来设计，即不管干扰源实际在哪个位置，也不管第二次测向误差取多少，我们都希望第二次观测后剩下的定位区域尽量小。

我们在选点时提出两个条件：

1. 先保证第二检测点一定能收到信号；

2. 在保证接收的前提下，让两次视线方向尽量叉开，交会角尽量大。

两次视线越接近平行，定位区域就会越细长；只有第二次观测方向与第一次观测方向形成明显夹角，才能把可能的干扰源位置划入区域。

6.3 无穷接收约束的三点化简

要解决第二点一定收得到信号的问题。因为平方距离关于角度和径向参数都具有凸性，所以连续无穷约束只需检查三个关键位置：扇形顶点 S_1 和外圆弧的两个端点 $G_\pm$。

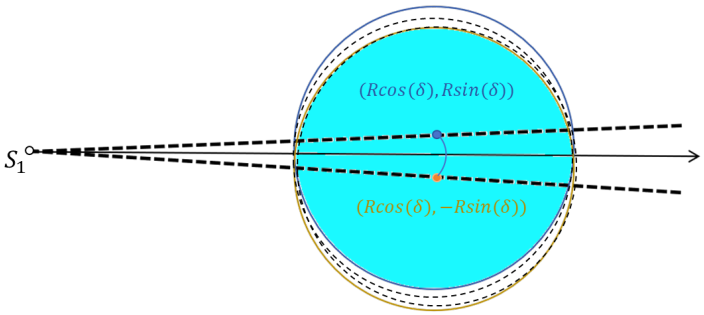


图 **4**　圆弧端点形成的透镜形区域

在以 u_1, n_1 为基的局部坐标系中，写 $S_2 - S_1 = au_1 + bn_1$。先固定实际接收半径 ρ，暂不考虑圆域 $\mathcal{D}$ 的截断。第一次示向扇区外侧圆弧上的目标位置为

$$G(\varphi) = S_1 + \rho(\cos\varphi\, u_1 + \sin\varphi\, n_1), \qquad |\varphi| \le \delta.$$

对圆弧上所有相容目标都保证接收，等价于要求 $S_2 \in \bigcap_{|\varphi|\le\delta} B(G(\varphi), \rho)$。利用约束平方后的凸性，无穷多个圆盘的交集可化简为两个端点圆盘的交集：

$$\mathcal{L}(\rho) = B(G_+(\rho), \rho) \cap B(G_-(\rho), \rho) = \left\{ S_1 + au_1 + bn_1 : a^2 + b^2 \le 2\rho(a\cos\delta - |b|\sin\delta) \right\}. \tag{11}$$

其中 $G_\pm(\rho) = G(\pm\delta)$。该式只处理了扇区外侧圆弧端点 $t = \rho$；另一个径向端点 $t = 0$ 由后续顶点圆盘约束处理。详细化简过程见附录。

固定 φ 后，$\|S_2 - G\|^2$ 关于径向距离 t 为凸二次函数，在 $0 \le t \le \rho$ 上的最大值位于 $t = 0$ 或 $t = \rho$。后一端点已由式 (11) 处理；前一端点要求 $S_2 \in B(S_1, \rho)$。当 ρ 在 $[\rho_-, \rho_+]$ 内变化时，只需取最小半径 ρ_-，因为满足式 (11) 的候选点上 $a\cos\delta - |b|\sin\delta \ge 0$，故两类约束均随 ρ 增大而放宽。于是解析鲁棒接收域为

$$\mathcal{C}_{\text{rec}} = \left[B(S_1, \rho_-) \cap B(G_+(\rho_-), \rho_-) \cap B(G_-(\rho_-), \rho_-) \right] \setminus \{S_1\}. \tag{12}$$

等价地，令 $S_2 = S_1 + au_1 + bn_1$，只需检验

$$(a, b) \ne (0, 0), \qquad \begin{cases} a^2 + b^2 \le \rho_-^2, \\ a^2 + b^2 \le 2\rho_-(a\cos\delta - |b|\sin\delta). \end{cases}$$

该域是第二检测点的保证接收候选范围。由于实际定位区域还与 $\mathcal{D}$ 相交，该条件对原问题是保守的：域内一定能接收，域外则不能据此断言无法接收。

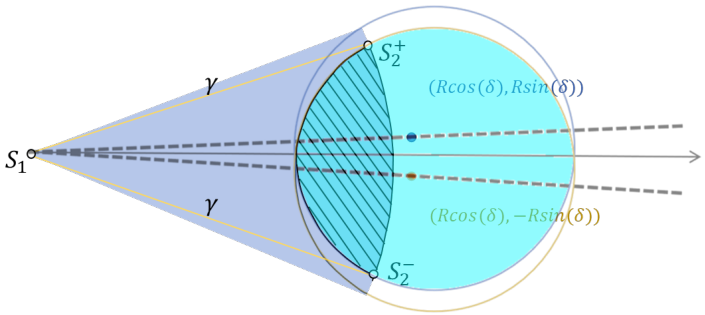


图 **5**　加入扇形顶点对应的圆盘（斜线阴影部分为鲁棒接收域）

6.4 最坏交会角与解析基准点

设从 S_1, S_2 指向目标的两条视线交会角为 α。由三角形面积关系，

$$|\sin\alpha| = \frac{|b\cos\varphi - a\sin\varphi|}{\|G - S_2\|}.$$

对 $|\varphi| \le \delta$ 取最坏情形，并利用 $\|G - S_2\| \le \rho \le \rho_+$，可得

$$|\sin\alpha| \ge \underline{\sigma}(a,b) := \frac{|b|\cos\delta - a\sin\delta}{\rho_+}, \tag{13}$$

在 $\mathcal{C}_{\text{rec}}$ 中最大化该下界，等价于最大化 $|b|\cos\delta - a\sin\delta$；最大值在两个等半径圆盘的外侧交点处取得。令 $\gamma = \pi/3 - \delta$，两个对称的解析基准点为

$$S_2^{\pm} = S_1 + \rho_-\big(\cos\gamma\, u_1 \pm \sin\gamma\, n_1\big). \tag{14}$$

代入 $\rho_- = 1000$ m、$\delta = 1°$，即从第一次中心示向线向两侧分别偏转 $59°$，移动 1000 m。这里的最优性是指在保守接收域内使式 (13) 的下界最大，并非直接求得在所有可行位置上的全局最优解。

6.5 两次观测后的定位直径界

引理（两次示向的直径界）。设两次检测点为 S_a, S_b。若定位区域中的任意可行点到两个检测点的距离均不超过 R，且从这两个检测点指向任意可行点的视线满足 $|\sin\alpha| \ge \sigma > 0$，则两次示向后的定位区域 Ω' 满足

$$D(\Omega') \le \frac{2R\sin(2\delta)}{\sigma}. \tag{15}$$

证明见附录 2.5。

在式 (14) 的基准点处，式 (13) 给出统一交会角下界

$$\sigma_* = \frac{\rho_-}{\rho_+}\sin\!\Big(\frac{\pi}{3} - 2\delta\Big)\,. \tag{16}$$

第二次观测后的任意可行点到 $S_1, S_2^{\pm}$ 的距离均不超过 ρ_+。将 $R = \rho_+$、$\sigma = \sigma_*$ 代入式 (15)，得到

$$D(\Omega_2) \le \frac{2\rho_+^2 \sin(2\delta)}{\rho_- \sin(\pi/3 - 2\delta)} \approx 185.19\ \mathrm{m} \quad (\delta = 1^\circ). \tag{17}$$

该上界适用于有界测角误差下的所有相容源位置；圆域截断和后续观测还可能进一步缩小实际定位区域。

6.6 结论

第二检测点的保证接收候选区域为

$$\mathcal{C}_{\mathrm{rec}} = B(S_1, 1000) \cap B(G_+, 1000) \cap B(G_-, 1000),$$

最优解析基准点为

$$S_2^{\pm} = S_1 + 1000\, R_{\pm 59^\circ} u_1,$$

即从首次中心示向线向两侧偏转 59°、前进 1000 m；在该点完成第二次示向后，定位直径满足

$$D(\Omega_2) \le 185.19\ \mathrm{m}.$$

七、 问题三：全向干扰源的搜索与清除策略

7.1 七点搜索与四次示向的保证

七点负责发现和判空

在目标圆域的圆心及半径为 1130 m 的正六边形六个顶点设置兜底检测点，如图 6。每个点的 1000 m 保证接收范围共同覆盖半径 1800 m 的目标区域，因此一个真实存在的全向源，必定能在至少一个兜底点被发现。对某个频道，只有七点全部无信号才判其不存在；一旦收到示向，该频道便转入定位，不必继续完成它的七点检测。外围半径的选择和覆盖证明见附录 2.2，正文保留搜索保证的核心条件：

$$\mathcal{D} \subseteq B(O, 1000) \cup \bigcup_{j=1}^{6} B(P_j, 1000). \tag{18}$$

四次示向负责已发现源的定位

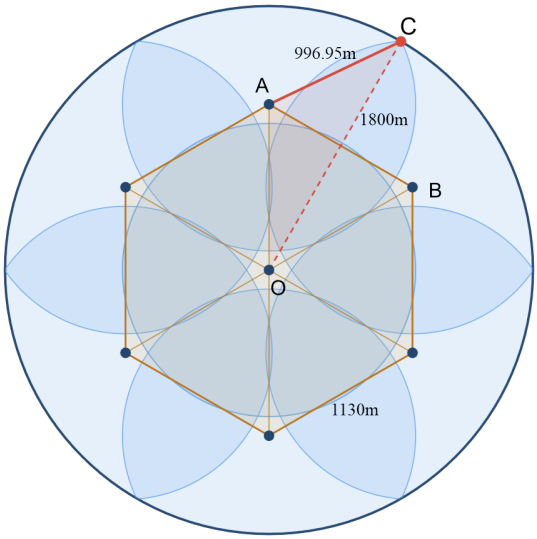


图 **6**　七点搜索：中心点和六个外围点的保证接收范围覆盖目标区域

第一条示向确定一条狭长的可能位置区域；第二点按问题二的鲁棒选点方法选择，保证能收到信号并从侧面交会。第三点取当前可能区域的最小覆盖圆圆心，第四点再从侧面补测。按这条保证路径执行，最多四次示向后便能前往清除位置；若中途已收到近距离信号或已达到清除条件，则提前清除。完整选点与误差界证明见问题二及附录 2.5。这里的“四次”只约束保证路径上的示向，不限制搜索检测或顺路观测的总次数。

7.2 四观测选点与扇形递推收缩

第二检测点由式 (14) 的两个对称基准点中选取：两点具有相同的最坏保证，可按实际可达性与移动代价择一。取得第二次示向后，按问题一的半平面裁剪更新定位区域 Ω_2，再计算其直径和最小覆盖圆。若覆盖圆半径已不超过 20 m，便直接进入光学定位与清除；否则按图 7 的两阶段几何构造继续选点。

图的左侧是前两次示向的交会区域，其直径已由式 (17) 控制在 185.19 m 以内。第三点 S_3 取 Ω_2 的最小覆盖圆圆心，记覆盖半径为 L。由平面 Jung 定理 [4]，

$$\|G - S_3\| \le L = R_{\mathrm{MEC}}(\Omega_2) \le \frac{D(\Omega_2)}{\sqrt{3}} \le 106.92\ \mathrm{m} < \rho_- = 1000\ \mathrm{m}. \tag{19}$$

所以第三点一定能接收信号。若尚未进入近距离清除分支，第三次示向 u_3 与该覆盖圆相交后，使新的区域满足 $\Omega_3 \subseteq \mathcal{W}(S_3, u_3, L)$。这一步把不规则的 Ω_2 重新外包为以 S_3 为顶点、径向界为 L 的窄扇形，与第一次示向后的几何结构相同。

图的右侧展示了同一侧向选点构造的再次使用。记 R_β 为逆时针旋转 β 的算子；从

u_3 向一侧偏转 $\gamma = \pi/3 - \delta = 59°$，移动 L 到

$$S_4 = S_3 + LR_{\pm(\pi/3-\delta)}u_3. \tag{20}$$

真实源方向与移动方向的夹角在 58°–60° 内。由余弦定理可知 $\|G - S_4\| \le L$，故第四点也必能接收；同一夹角又保证第三、第四条视线不会近乎平行。于是再次应用前述两次示向直径界，得到一次标准扇形收缩

$$D(\Omega_4) \le \lambda_\delta L, \qquad \lambda_\delta = \frac{2\sin(2\delta)}{\sin(\pi/3 - 2\delta)}. \tag{21}$$

在 $\delta = 1°$ 时，$\lambda_\delta \approx 0.0823$，故 $D(\Omega_4) \le 8.80\,\mathrm{m}$，进而 $R_{\mathrm{MEC}}(\Omega_4) \le 5.08\,\mathrm{m} < 20\,\mathrm{m}$。因此从首次发现全向源起，至多四次示向即可保证进入光学清除范围；若途中已满足近距离清除条件，观测次数更少。实际选点使用当前 Ω_2 的最小覆盖圆半径；完整的几何推导见附录 2.5。

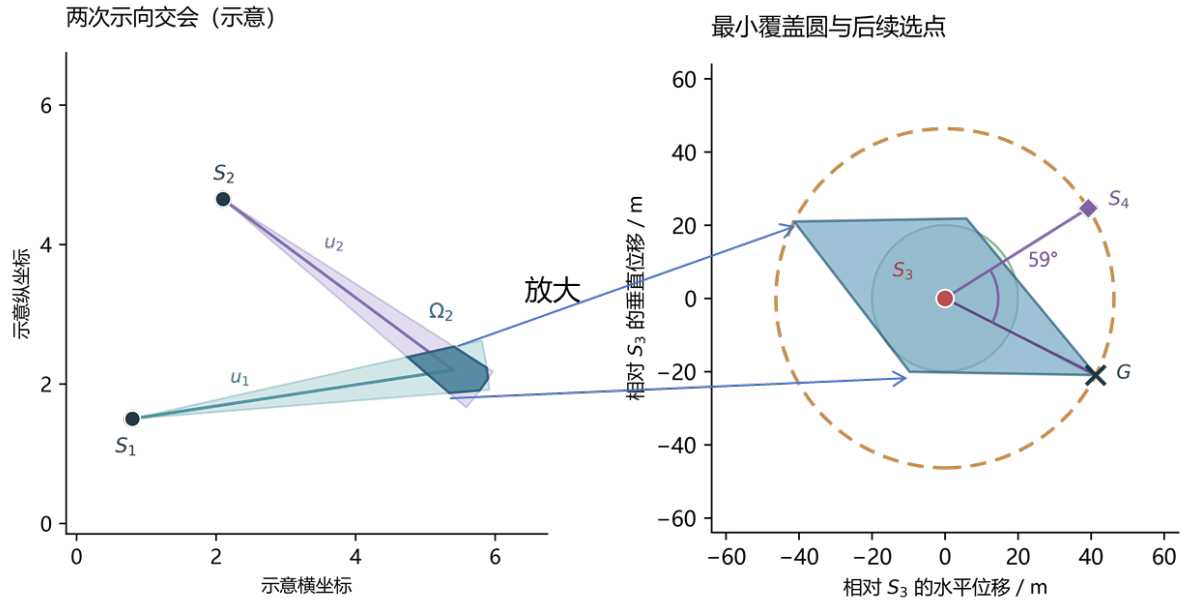


图 7　四次观测的两阶段定位构造：左侧由前两次示向形成 Ω_2，右侧由最小覆盖圆圆心 S_3 和侧向点 S_4 完成再次收缩

7.3 Baseline 与优化版的执行流程

两个方案基本流程相同。每次只选择下一处停靠点；机器狗到站取得反馈后，更新各频道状态和待办表，再规划下一步。因此不必先走完七点，也不必发现一个源就连续做完四次示向。图 8 给出共同流程及优化版增加的动作。

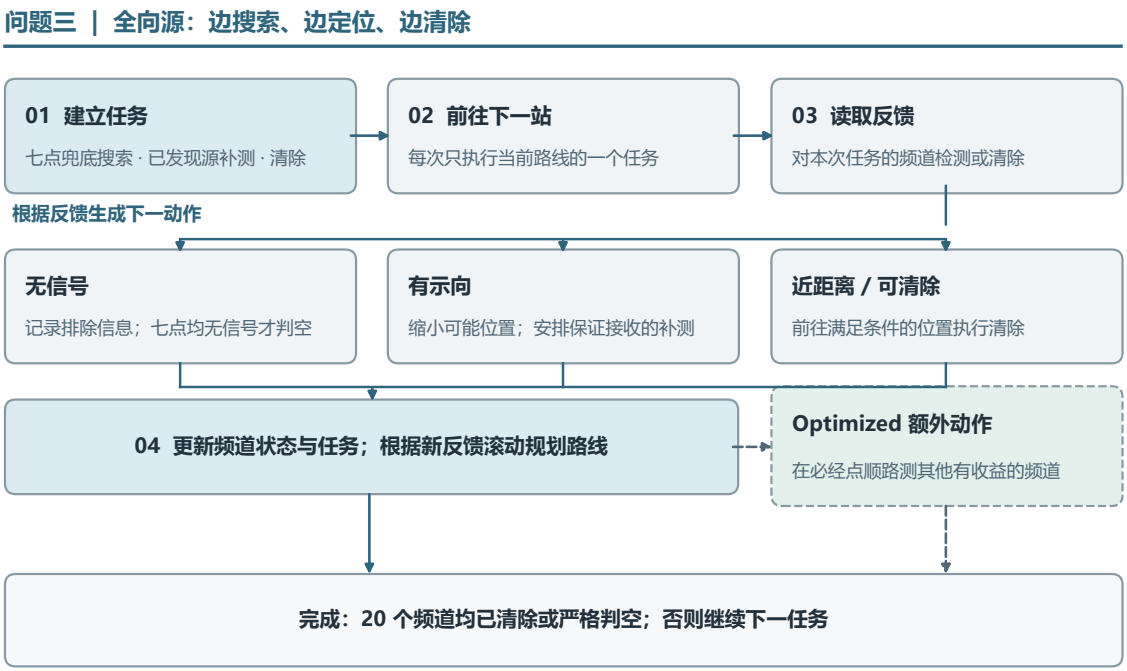


图 **8**　问题三的在线执行流程。虚线框是优化方案增加的顺路观测。

Baseline 的反馈处理

兜底点无信号时，记录该频道已排除的区域；七点都无信号才判空。有示向时，合并已有读数以缩小可能位置，并按四次示向保证路径安排下一补测点。可能位置已被半径 20 m 的圆覆盖时，机器狗前往圆心清除；收到近距离反馈时直接清除。清除失败则换点取得新示向，不在原位置无信息地重复尝试。清除判据仍与问题一的最小覆盖圆一致：

$$R_{\mathrm{MEC}}\left(\Omega_c^{\mathrm{p}}\right) \le 20\ \mathrm{m}. \tag{22}$$

其中 Ω_c^{p} 是该频道可能位置的保守外包。示向角域、接收距离和历次无信号信息都参与更新，计算时保持外包不漏掉相容位置；三段判据见附录 2.4。

优化版的新增动作

机器狗在原定停靠点完成任务后，顺便检查其他频道是否值得检测：优先考虑已发现且能从新方向缩小定位范围的源，再考虑可增加搜索覆盖的未知频道。每站最多增加四次机会观测，其中未知频道最多两次；这些观测不产生新的移动目的地。其示向可提前缩小可能位置，若已满足清除条件便直接生成清除任务；无信号反馈不能代替七点判空，机会观测也不占用四次示向保证路径的名额。Baseline 不执行这一步，其余反馈处理和滚动路线规则相同。

路线如何更新

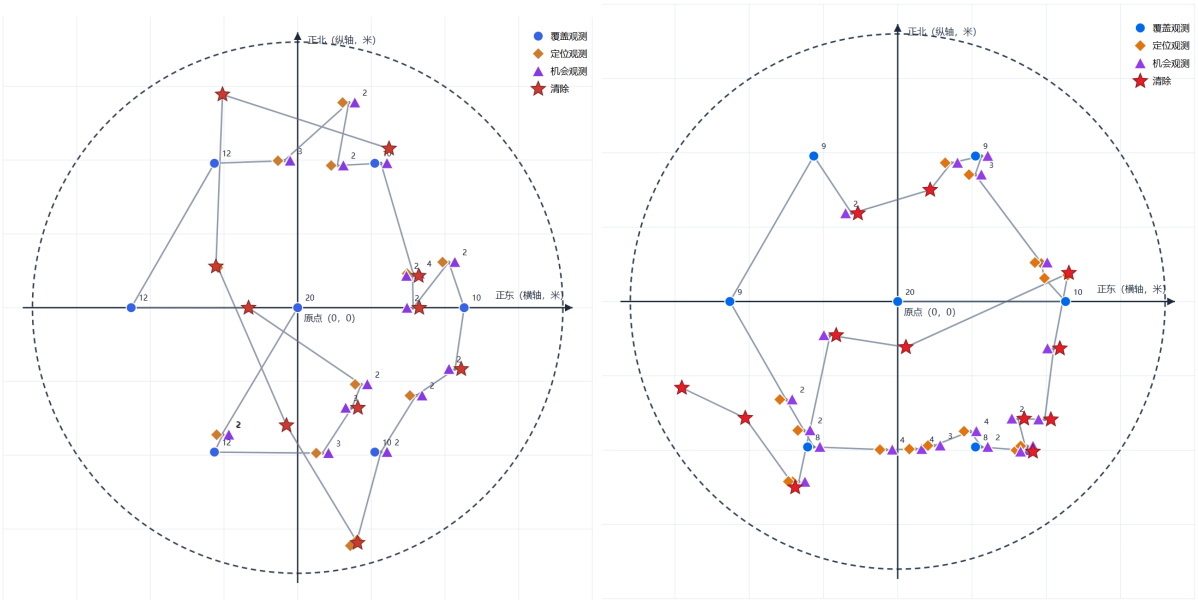
两版都优先缩短移动距离：先为尚未完成的任务生成开放路线，再插入新发现目标的补测点和清除点，消除明显折返。机器狗只执行路线中的第一项，到站后重新规划；

这样多频道任务可交错进行。它是一条随反馈调整的可行路线，并非预先知道全部源位置后的全局最短路线。

7.4 演练与三次正式测试结果

7.4.1 演练结果

正式测试前，先在固定种子的本地交互器中作同场景配对演练。种子 20260911、13 个干扰源的场景下，Baseline 和加入零绕行机会观测的 Anytime V1 都清除全部目标，清除失败次数均为零。表 2 保留该场景的逐项结果；图 9 展示一次演练的实际搜索路径。该交互器让同一频道、同一位置的示向误差固定且不依赖查询顺序，可用于控制策略比较中的场景差异，但不等同于官方模拟器。



(a) (b)

图 **9**　两次演练的实际搜索路径

表 **2**　同一场景下两种策略的配对实验结果

指标	原版	Anytime V1	变化
清除率	13/13	13/13	—
虚拟总时间 / s	5660.35	4315.12	−23.77%
平均定位清除时间 / (s/个)	435.41	331.93	−23.77%
移动距离 / m	22256.73	17625.59	−20.81%
覆盖往返额外距离 / m	9503.71	2631.38	−72.31%

另对 Baseline 与 Anytime V1 各完成 20 次演练。逐动作日志按移动、检测、换频和

清除四项复算，分项之和与模拟器记录的虚拟总时间一致。表 3 给出两组样本的主要均值；两组场景并未逐一配对。

表 **3**　两种策略各 **20** 次演练的主要结果

指标	Baseline	Anytime V1
虚拟总时间均值 / s	5130.88	4551.35
平均定位清除时间 / (s/个)	397.34	358.65
平均移动距离 / km	19.75	18.35
平均检测次数	188.05	138.05
平均清除个数	13.15	12.80

7.4.2 三次正式测试

正式测试在官方模拟器中进行，目标数量、频道、位置及接收半径在运行前均不可读取。表 4 按题目要求列出清除个数、平均定位清除时间和程序现实运行时间；平均定位清除时间按该次定位清除总时间除以清除个数计算。导出的三份日志保持原文件名并列入支撑材料。

表 **4**　问题三正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 / s	程序运行时间 / s
3ATD-ZRKR-VVQK-2AM8	14	346.47	1.4
8S4S-PACC-HMNX-WHC6	10	361.17	1.48
638X-M9XC-79RX-V673	13	386.19	2.34

7.5 策略分析

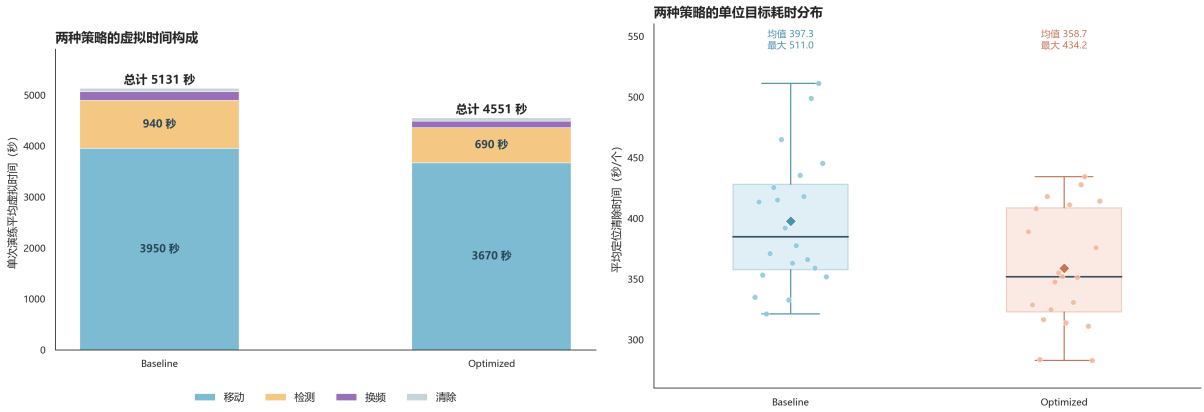
7.5.1 时间构成与稳健性

为辨别优化收益来自哪类开销，按逐动作日志把虚拟总时间分为移动、检测、换频和清除四项；各项按题设速度与操作耗时复算。两组各 20 次的分项均值见表 5。

表 **5**　**20** 次演练虚拟时间均值的分解

耗时项目	Baseline / s	Anytime V1 / s	均值差 / s	差值占比
移动	3950.03	3669.80	280.23	48.35%
检测	940.25	690.25	250.00	43.14%
换频	174.85	127.30	47.55	8.20%
清除	65.75	64.00	1.75	0.30%
虚拟总时间	5130.88	4551.35	579.53	100%

两组移动时间分别占 76.99% 和 80.63%，是最大耗时项；Anytime V1 总时间均值低 579.53 s，其中移动与检测的均值差合计占 91.49%。这些百分比只描述本批样本，不能解释为严格因果贡献。



(a) 虚拟时间构成　　**(b)** 单位目标耗时分布

图 **10**　两种策略各 **20** 次演练的时间构成与单位目标耗时分布

以每次演练的虚拟总时间除以该次清除个数，得到单位目标耗时，分布见表 6。

表 **6**　各 **20** 次演练的单位目标耗时分布

指标	Baseline	Anytime V1
均值 / (s/个)	397.34	358.65
样本标准差 / (s/个)	54.26	48.07
变异系数	13.66%	13.40%
第 90 百分位 / (s/个)	468.06	418.74
样本最大值 / (s/个)	510.98	434.16
超过 450 s/个的次数	3/20	0/20

Anytime V1 的第 90 百分位、最大值及超过 450 s/个的次数均较低，说明较慢案例有所改善；两组变异系数接近，不能据此声称相对波动显著下降。按清除个数分层，Anytime V1 在共同出现的数量层内单位目标耗时均值均较低，但各层样本仅 2–5 次，只作为旁证。

7.5.2 适用边界与改进方向

七点覆盖、有效误差角和最小覆盖圆清除判据均取保守一侧，机会观测阈值只影响效率；四次观测上界仅适用于全向源和有界示向误差场景。现有演练并非逐场景配对，三次正式测试也对应不同案例，后续应增加同场景重复配对与阈值消融，以更稳定地估计各模块效率贡献。

八、 问题四：定向干扰源的搜索与清除策略

8.1 25 点搜索与三角形判空决策

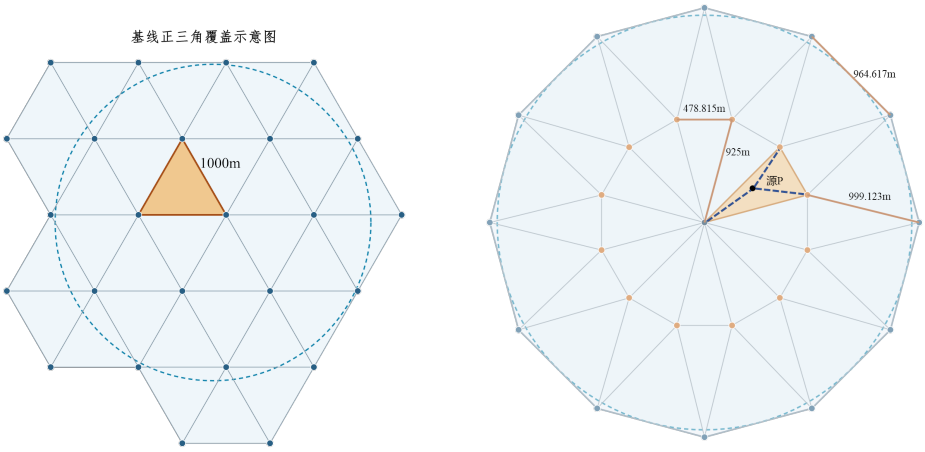
机器狗要从不同方向围住可能的源。把目标区域分成覆盖三角形，并使每个三角形的三个顶点都处在该三角形内任意源位置的 1000 m 保证接收范围内。这样无论源朝向哪里，至少有一个顶点在其辐射正面；只有同一频道在一个三角形的三个顶点都无信号，才能排除该三角形。全部覆盖三角形均被排除后，该频道才可判空。几何核验归结为一条布点条件：

$$\mathcal{D} \subseteq \bigcup_{T\in\mathcal{T}} T, \qquad \max_{a,b\in V(T)} \|a-b\| \le 1000\ \text{m} \quad (T\in\mathcal{T}). \tag{23}$$

其中 $\mathcal{T}$ 为覆盖三角形集合，$V(T)$ 为三角形的三个顶点。对频道 c，记 $n_c(p)=1$ 表示严格检测点 p 上无信号；一个三角形被排除的判据为

$$e_c(T) = \prod_{p\in V(T)} n_c(p) = 1. \tag{24}$$

这两条式子只服务于搜索保证和判空决策；连续位置与未知朝向的证明放在附录 2.1。顺路补测得到的无信号不充当三角形顶点证据，已发现定向源后的无信号也不能简单解释为距离过远。



(a) 正三角网格：**Baseline**　　**(b)** 中心双环：优化版

图 **11**　问题四的两种 **25** 点严格搜索布置。两者均保证不漏检，优化版减少目标圆外的外扩与固定路线长度。

Baseline 的布点：平移正三角网格得到 25 个严格检测点和 33 个覆盖三角形；预存路线长 24.173 km，其中 15 个点在目标圆外，最远外扩约 794 m。

优化版的布点：仍使用 25 点，但改为中心点、12 个内环点和 12 个外环点，由 36 个三角形覆盖目标区域；最远外扩约 63.5 m，预存搜索路线长 17.802 km。两套布点都

逐一核验式 (23)，差别在于优化版把必经检测点和路线收得更紧。布局见图 11；半径、三角剖分和路线的构造细节见附录 2.1。

8.2 Baseline 与优化版的执行流程

两版使用同一套反馈处理和定位清除程序，只替换严格搜索的点位、三角形证书及预存路线。机器狗沿对应路线访问仍需证据的点；在每站先检测尚未确认的频道，再对已发现而正在定位的频道做有限次顺路观测。图 12 给出共同执行流程。

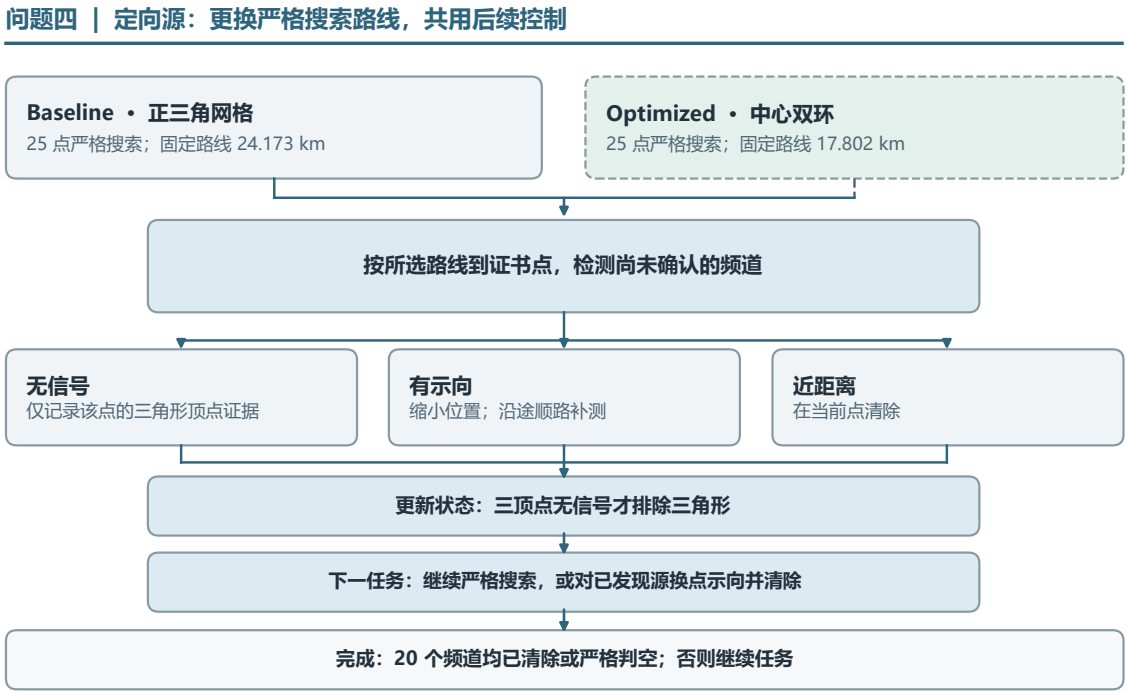


图 **12**　问题四两版共有的执行流程。虚线框仅表示选用不同的预存点位与路线；后续反馈、顺路观测、定位及清除规则相同。

到点后如何处理反馈。对尚未发现的频道，严格检测点无信号只更新相应三角形的顶点记录；全部三角形被排除时才判不存在。一旦收到示向，停止该频道尚未必要的搜索检测，按方向和接收距离缩小可能位置。每个必经点还可对至多四个正在定位、此前未在该点测过的频道顺路补测；这一步提高信息利用率，但不参与严格判空。

搜索后如何定位和清除。优先处理距当前位置较近的已发现频道。若可能位置已能被半径 20 m 的圆覆盖，就前往其圆心清除；近距离反馈则在当前点清除。否则围绕当前可能区域换方向示向，每次反馈后重新估计位置，再决定下一检测点。定向源的无信号可能来自背向，不能像问题三那样直接删去一个接收圆；这里也没有固定四次示向的保证。清除失败则退回定位，不把频道误记为完成。

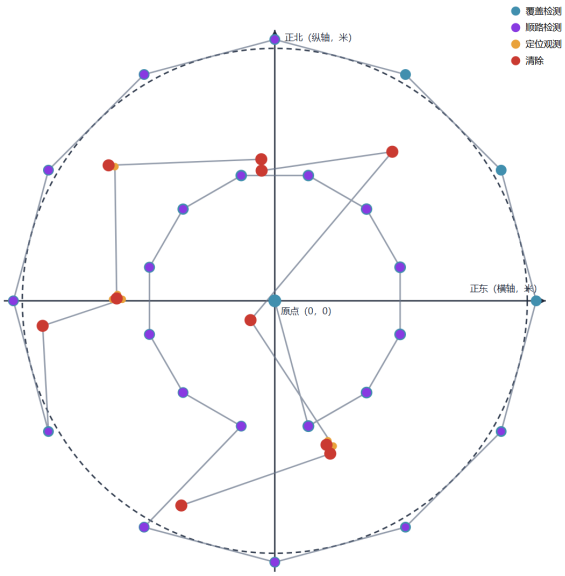


图 **13**　中心双环方案一次演练的实际搜索路径

8.3 正式测试结果

三次正式测试的结果按表 7 的格式报告，表中数据以正式测试导出的日志为准。

表 **7**　问题四正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 / s	程序运行时间 / s
DMDM-T89S-56RT-CKCF	11	633.01	3.31
NWPP-53U8-A72J-TVCJ	15	504.14	2.97
W7MP-4JU4-5BA4-Y8C8	14	578.26	3.51

三次测试的清除源数分别为 11、15、14，平均定位清除时间为 633.01 s、504.14 s、578.26 s，程序运行时间均约 3 s，远低于 20 分钟上限。问题四的这三次平均定位清除时间高于问题三的三次测试（约 346–386 s），体现了更密的严格搜索和更高的定向源定位难度。

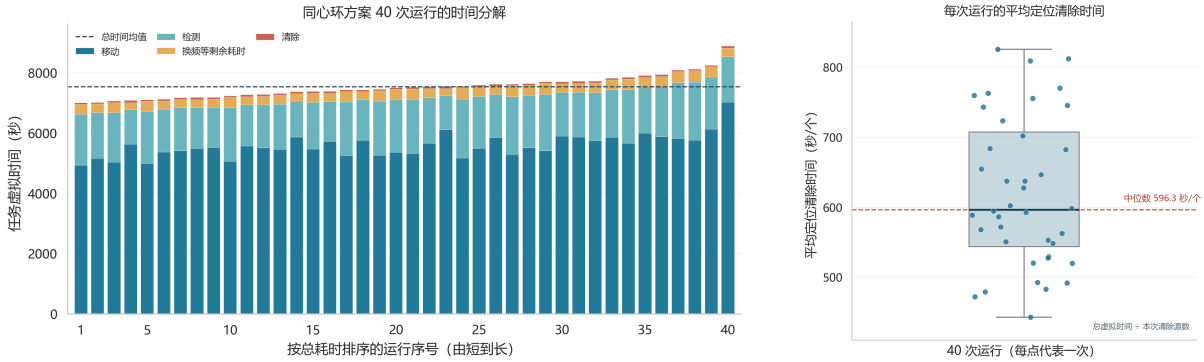
8.4 策略分析

8.4.1 稳健性分析

基于同心环方案的 40 次演练，所有频道均完成清除或判空，清除失败为 0；原始参数下的模拟任务总耗时均值为 7535.96 s。

去掉最高和最低各 4 次后，10% 截尾均值为 7497.21 s，仅比原均值低 0.51%；逐一删除任意一次运行所得均值位于 7501.27–7549.49 s；按清除源数分组的平均总耗时分别为 7524.94 s 和 7546.97 s；对 40 次运行进行 20000 次有放回重采样，均值的 95% 百分

位区间为 7425.25–7659.84 s。这些统计说明总耗时均值并非由少数极端案例主导。图 14 展示耗时构成与单次平均定位清除时间的分布：移动占虚拟总时间的 74.10%，是主要成本，也为下文的速度敏感性结果提供解释。



(a) 虚拟时间构成　　**(b)** 平均定位清除时间分布

图 **14**　同心环方案 **40** 次演练的耗时结构：左图是虚拟时间按移动、检测、清除与换频的分解，右图是每次运行的平均定位清除时间分布

8.4.2 成本参数敏感性分析

保持 40 次已经记录的轨迹、动作次数和完成状态不变，仅改变成本参数并重算虚拟时间，结果见表 8。

表 **8**　同心环方案固定轨迹下的成本参数敏感性

成本参数	平均重计费总时间 / s	相对原均值变化 / s
原始参数	7535.96	0
速度降低 10%	8156.45	+620.50
速度提高 10%	7028.28	−507.68
检测耗时降低 20%	7219.86	−316.10
检测耗时提高 20%	7852.06	+316.10

表 8 显示，速度降低或提高 10% 分别使平均总时间增加 620.50 s、减少 507.68 s；两侧变化不对称，源于移动耗时与速度成反比。检测耗时改变 20% 则使总时间对称地变化 316.10 s。结合图 14 中移动占比最高的结果可知，在当前任务与轨迹下，改善行进效率比单纯缩短检测操作更能降低总耗时。这里比较的是固定轨迹的成本响应。

九、 模型的评价与推广

模型优点

本文四个问题形成了统一的几何建模框架：问题一用半平面裁剪和旋转卡壳给出定位域直径及覆盖判定；问题二通过连续接收约束的三点化简，得到保证接收的鲁棒候选区域和解析基准点；问题三进一步给出七点完备搜索与四观测有限步定位保证；问题四用三角覆盖证书处理定向源背面漏检问题，理论推导和算法实现形成了较完整的闭环。

模型的可计算性较强。问题一、二的核心几何算法均为线性或低阶多项式复杂度，问题三、四的程序运行时间均控制在数秒以内，能够满足实际测试环境的时间要求。

模型经过了演练、正式测试、稳健性分析和参数敏感性分析，不仅报告了清除个数和平均定位清除时间，还对移动、检测、换频、清除等时间构成进行了分解，说明优化方向主要来自紧凑布点和滚动路线。

模型不足

问题四的主动定位阶段仍采用滚动启发式策略，没有给出固定步数的定位保证；搜索布点的 25 个点和 17.802 km 路线是可行上界，不是严格意义下的全局最优。此外，受正式测试样本数量和模拟器随机性限制，不同策略之间难以完全配对比较。示向误差按有界误差处理，虽然保证了保守性，但也会使结果偏保守。

模型推广

本文的定位和搜索框架可以推广到多站测向、无人机编队搜索以及带遮挡区域的传感器网络任务中。将问题四的三角覆盖证书替换为更一般的凸区域剖分后，还可以处理非圆形区域或含障碍物的搜索场景；问题三的在线滚动调度思想也可用于其他搜索定位类任务。

AI 工具使用说明

依据竞赛要求，在参考文献之前声明 AI 工具的使用情况。本论文在完成过程中使用了 AI 辅助工具 Codex，主要用于协助完成数据统计、结果整理、文本润色与公式规范化。所有模型假设、算法选择、参数设置与结果结论均由团队成员讨论确认并经代码与数据校验，AI 仅作为辅助工具，未替代团队的建模决策。

参考文献

[1] Preparata F P, Shamos M I. Computational Geometry: An Introduction[M]. New York: Springer-Verlag, 1985. DOI: 10.1007/978-1-4612-1098-6.

[2] Toussaint G T. Solving geometric problems with the rotating calipers[C]//Proceedings of IEEE MELECON. Athens, 1983: A10.02/1–4.

[3] Welzl E. Smallest enclosing disks (balls and ellipsoids)[C]//New Results and New Trends in Computer Science. Lecture Notes in Computer Science, Vol. 555. Berlin: Springer, 1991: 359–370. DOI: 10.1007/BFb0038202.

[4] Jung H. Über die kleinste Kugel, die eine räumliche Figur einschließt[J]. Journal für die reine und angewandte Mathematik, 1901, 123: 241–257. DOI: 10.1515/crll.1901.123.241.

[5] Torrieri D J. Statistical theory of passive location systems[J]. IEEE Transactions on Aerospace and Electronic Systems, 1984, AES-20(2): 183–198. DOI: 10.1109/TAES.1984.310439.

[6] Ben-Tal A, El Ghaoui L, Nemirovski A. Robust Optimization[M]. Princeton: Princeton University Press, 2009. DOI: 10.1515/9781400831050.

[7] Choset H. Coverage for robotics—A survey of recent results[J]. Annals of Mathematics and Artificial Intelligence, 2001, 31(1–4): 113–126. DOI: 10.1023/A:1016639210559.

[8] Ma H, Liu Y. On coverage problems of directional sensor networks[C]//Mobile Ad-hoc and Sensor Networks (MSN 2005). Lecture Notes in Computer Science, Vol. 3794. Berlin: Springer, 2005: 721–731. DOI: 10.1007/11599463_70.

附录 A　支撑材料文件列表

表 **9**　支撑材料文件列表

文件名	类型	说明
AI 工具使用说明与交互记录.pdf	文档	AI 工具使用说明与交互记录
README.md	说明	支撑材料目录、代码运行和绘图方法说明
代码/Q3/baseline/__init__.py	程序	问题三 baseline 模块初始化文件
代码/Q3/baseline/controller.py	程序	问题三 baseline：搜索、定位与清除控制器
代码/Q3/baseline/runner.py	程序	问题三 baseline：策略运行入口
代码/Q3/common/__init__.py	程序	问题三 common 模块初始化文件
代码/Q3/common/active_sensing.py	程序	问题三 common：主动观测点选择
代码/Q3/common/communication.py	程序	问题三 common：官方接口适配
代码/Q3/common/diagnostics.py	程序	问题三 common：运行指标与诊断记录
代码/Q3/common/geometry.py	程序	问题三 common：几何计算
代码/Q3/common/localization.py	程序	问题三 common：示向交会定位
代码/Q3/common/models.py	程序	问题三 common：状态与数据结构
代码/Q3/common/plotting.py	程序	问题三 common：运行路径绘图
代码/Q3/common/q3_path_planner.py	程序	问题三 common：覆盖与路径规划
代码/Q3/optimized/__init__.py	程序	问题三 optimized 模块初始化文件
代码/Q3/optimized/controller.py	程序	问题三 optimized：搜索、定位与清除控制器
代码/Q3/optimized/runner.py	程序	问题三 optimized：策略运行入口
代码/Q3/start_baseline.bat	脚本	问题三基线版启动脚本
代码/Q3/start_optimized.bat	脚本	问题三优化版启动脚本
代码/Q4/baseline/__init__.py	程序	问题四 baseline 模块初始化文件
代码/Q4/baseline/runner.py	程序	问题四 baseline：策略运行入口
代码/Q4/common/__init__.py	程序	问题四 common 模块初始化文件
代码/Q4/common/geometry.py	程序	问题四 common：几何计算
代码/Q4/common/planner.py	程序	问题四 common：搜索覆盖、定位规划及诊断绘图
代码/Q4/optimized/__init__.py	程序	问题四 optimized 模块初始化文件
代码/Q4/optimized/runner.py	程序	问题四 optimized：策略运行入口
代码/Q4/run_baseline.bat	脚本	问题四基线版启动脚本
代码/Q4/run_optimized.bat	脚本	问题四优化版启动脚本
代码/shared/__init__.py	程序	共享模块初始化文件
代码/shared/official_client.py	程序	两题共用的官方接口与日志客户端
代码/绘图部分/Q3/README.md	说明	问题三绘图数据和运行说明
代码/绘图部分/Q3/figures/图 1_时间构成.pdf	图件	问题三图 1_ 时间构成（PDF 图）
代码/绘图部分/Q3/figures/图 1_时间构成.png	图件	问题三图 1_ 时间构成（PNG 图）
代码/绘图部分/Q3/figures/图 1_时间构成.svg	图件	问题三图 1_ 时间构成（SVG 图）
代码/绘图部分/Q3/figures/图 2_稳健性.pdf	图件	问题三图 2_ 稳健性（PDF 图）
代码/绘图部分/Q3/figures/图 2_稳健性.png	图件	问题三图 2_ 稳健性（PNG 图）

表 **9**　支撑材料文件列表（续）

文件名	类型	说明
代码/绘图部分/Q3/figures/图 2_稳健性.svg	图件	问题三图 2_ 稳健性（SVG 图）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185102.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185115.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185128.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185142.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185155.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185208.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185220.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185234.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185247.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185301.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185318.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185348.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185403.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185416.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185429.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185443.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185455.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185521.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185535.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_metrics_20260912-185559.json	数据	问题三优化版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_anytime_运行指标.csv	数据	问题三统计图输入数据

表 **9**　支撑材料文件列表（续）

文件名	类型	说明
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184454.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184511.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184525.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184539.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184553.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184609.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184622.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184640.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184657.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184712.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184727.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184741.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184755.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184822.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184837.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184850.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184903.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-184930.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-185011.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_metrics_20260912-185846.json	数据	问题三基线版演练运行指标（绘图输入）
代码/绘图部分/Q3/logs/robot_dog_运行指标.csv	数据	问题三统计图输入数据
代码/绘图部分/Q3/plot_comparison_figures.py	程序	问题三统计图生成脚本
代码/绘图部分/Q4/40 次绘图数据.csv	数据	问题四统计图输入数据

表 9　支撑材料文件列表（续）

文件名	类型	说明
代码/绘图部分/Q4/README.md	说明	问题四绘图数据和运行说明
代码/绘图部分/Q4/图 1_平均定位清除时间分布.pdf	图件	问题四图 1_ 平均定位清除时间分布（PDF 图）
代码/绘图部分/Q4/图 1_平均定位清除时间分布.png	图件	问题四图 1_ 平均定位清除时间分布（PNG 图）
代码/绘图部分/Q4/图 1_平均定位清除时间分布.svg	图件	问题四图 1_ 平均定位清除时间分布（SVG 图）
代码/绘图部分/Q4/图 2_逐次时间分解.pdf	图件	问题四图 2_ 逐次时间分解（PDF 图）
代码/绘图部分/Q4/图 2_逐次时间分解.png	图件	问题四图 2_ 逐次时间分解（PNG 图）
代码/绘图部分/Q4/图 2_逐次时间分解.svg	图件	问题四图 2_ 逐次时间分解（SVG 图）
代码/绘图部分/Q4/绘图.py	程序	问题四统计图生成脚本
正式测试日志文件/Q3/1/formal-p3-1-3ATD-ZRKR-VVQK-2AM8.jlog	日志	问题 3 第 1 次正式测试的模拟器原始日志
正式测试日志文件/Q3/2/formal-p3-2-8S4S-PACC-HMNX-WHC6.jlog	日志	问题 3 第 2 次正式测试的模拟器原始日志
正式测试日志文件/Q3/3/formal-p3-3-638X-M9XC-79RX-V673.jlog	日志	问题 3 第 3 次正式测试的模拟器原始日志
正式测试日志文件/Q4/1/formal-p4-1-DMDM-T89S-56RT-CKCF.jlog	日志	问题 4 第 1 次正式测试的模拟器原始日志
正式测试日志文件/Q4/2/formal-p4-2-NWPP-53U8-A72J-TVCJ.jlog	日志	问题 4 第 2 次正式测试的模拟器原始日志
正式测试日志文件/Q4/3/formal-p4-3-W7MP-4JU4-5BA4-Y8C8.jlog	日志	问题 4 第 3 次正式测试的模拟器原始日志

附录 B　补充推导

2.1　问题四三角证书与两种布点的核验

设源位于覆盖三角形 T 内。若三个顶点为 p_1, p_2, p_3，则源位置 x 可写为它们的凸组合，且对任一顶点 p_j 有

$$\|x - p_j\| \le \sum_{i=1}^{3} \lambda_i \|p_i - p_j\| \le \max_{a,b \in V(T)} \|a - b\| \le 1000\ \mathrm{m}.$$

因此三个顶点都在保证接收范围内。若源的辐射方向为 h，不可能使三个顶点同时严格落在辐射背面，否则其凸组合 x 也会落在自身背面。由此至少一个顶点可收到信号；三个顶点全无信号时，该三角形不存在该频道的源。全部证书三角形覆盖目标圆域，便得到正文式 (23) 和式 (24) 的严格判空依据。

Baseline 用 25 个正三角网格点构造 33 个覆盖三角形，证书边长为 1000 m；预存开

放路线长 24.173 km，最远检测点超出目标圆约 794 m。优化版用中心点、半径 925 m 的 12 个内环点和 12 个外环点。外环半径取

$$R_{\mathrm{o}} = \frac{1800}{\cos(\pi/12)} \approx 1863.497\ \mathrm{m},$$

使外环正十二边形包含目标圆，最大外扩约 63.5 m。优化版预存 36 个三角形，逐个核验面片铺满外环多边形且最长边不超过 1000 m；其固定开放路线长 17.802 km。25 点是本次离散候选中的选择，不是所有连续布点的最少点数证明；17.802 km 是已实现的可行路线长度，也不是该点集上最短路线的证明。

2.2 问题三七点覆盖半径的核验

七点由圆心和半径为 $a = 1130$ m 的正六边形六个顶点组成。六边形内部可分为六个全等正三角形；每个三角形的外接圆半径为 $a/\sqrt{3} \approx 652.41$ m，小于保证接收半径 1000 m。对六边形外、目标圆内的弓形区域，按最近外围顶点划分；最不利点位于目标圆边界与相邻外围顶点的角平分线上。若目标圆半径为 $R = 1800$ m，该点到最近外围顶点的距离满足

$$d^2 = R^2 + a^2 - 2Ra\cos 30^\circ.$$

令 $d \le 1000$ m，可得最小可行外围半径

$$a_{\min} = R\cos 30^\circ - \sqrt{1000^2 - R^2\sin^2 30^\circ} \approx 1122.96\ \mathrm{m}.$$

所用的 1130 m 比该下界多约 7 m。因此七个保证接收圆覆盖目标圆域，正文式 (18) 成立。若只用五个等角外围点，其半扇区角为 36°，所需边界垂距约 1058 m，超过保证接收半径；故在这种中心加等角外围点的布置中至少需要六个外围点。

2.3 问题一算法正确性的证明

半平面交与相容位置集合：每次观测对应的两个不等式分别把候选点限制在左边界射线的左侧与右边界射线的右侧，二者的交恰好是示向度落在 $[\theta_i - \delta, \theta_i + \delta]$ 内的角域。与目标圆域相交可得到仅利用示向信息的真实相容区域；与外切凸多边形相交则得到供多边形算法使用的保守外包。

旋转卡壳与直径：凸多边形的最远点对必是一对对踵点，而旋转卡壳在支撑线绕多边形一周的过程中枚举了全部对踵关系并逐一比较端点距离，故其最大值即直径。

直径圆覆盖的等价条件：任何覆盖圆的半径都不小于 $D/2$。若某一直径端点对的中点圆覆盖全部顶点，则它覆盖整个凸多边形并取到半径下界，故 $R_* = D/2$；反之，若存在直径等于 D 的覆盖圆，其半径即为 $D/2$，因而它本身就是最小覆盖圆；此时该圆

覆盖任一对直径端点，三角不等式必须取等，圆心必为该端点对的中点。因此固定任意一组直径端点并检验其对应圆已足够；若该圆不覆盖区域，就不存在其他圆心可使半径保持 $D/2$。

2.4 Jung 三段判据的推导

二维 Jung 定理给出

$$\frac{D}{2} \le R_* \le \frac{D}{\sqrt{3}},$$

其中下界即正文式 (7)。机器狗要保证进入光学精确定位范围，只需存在一个点使得到目标的距离不超过 20 m，即要求 $R_* \le 20$。把它代入上式可得：当 $D \le 20\sqrt{3} \approx 34.64$ m 时，$R_* \le D/\sqrt{3} \le 20$，无论定位区域呈什么形状都存在一个保证清除的驻留点；当 $D > 40$ m 时，$R_* \ge D/2 > 20$，无论区域呈什么形状都不可能仅凭当前的定位结果完成清除。只有 $20\sqrt{3} < D \le 40$ 这一段需要进一步计算 R_*，即正文式 (22)。

2.5 全向干扰源四观测定位定理

设示向误差满足 $|e| \le \delta$，本题 $\delta = 1^\circ$；实际接收半径未知但满足 $1000 = \rho_- \le \rho \le \rho_+ = 1500$ m。首次在 S_1 处获得中心示向 u_1，记其法向量为 n_1。定义以 S 为顶点、径向界为 R、半角为 δ 的标准扇形

$$\mathcal{W}(S, u, R) = \big\{ S + t(\cos\varphi\, u + \sin\varphi\, n) :\ 0 \le t \le R,\ |\varphi| \le \delta \big\}.$$

定理（全向干扰源四观测定位定理）。在上述条件下，从首次发现全向干扰源起，存在一个至多使用四次示向观测的确定性策略，使最终定位区域可被半径 20 m 的圆覆盖。

证明概要。令 $\gamma = \pi/3 - \delta = 59^\circ$，选择第二检测点

$$S_2 = S_1 + \rho_- R_{+\gamma} u_1,$$

其中 $R_{+\gamma}$ 表示逆时针旋转 γ。可以证明无论实际接收半径 ρ 取何值，均有 $\|G - S_2\| \le \rho$；同时两次视线交会角满足

$$\sin\alpha_{12} \ge \frac{\rho_-}{\rho_+} \sin\left(\frac{\pi}{3} - 2\delta\right).$$

以 ρ_+ 为径向界构造两次示向的闭定位外包 Ω_2，由直径界引理得到

$$D_2 = D(\Omega_2) \le \frac{2\rho_+^2 \sin(2\delta)}{\rho_- \sin(\pi/3 - 2\delta)} \approx 185.19 \text{ m}.$$

令 $L = R_{\mathrm{MEC}}(\Omega_2)$，并取 Ω_2 的最小覆盖圆圆心作为 S_3。该圆心由定位区域唯一确

定，因此策略具有确定性；由平面 Jung 定理，

$$L \le \frac{D_2}{\sqrt{3}} \le 106.92\ \mathrm{m} < 1000 \le \rho.$$

于是第三检测点必能收到信号；若此时已可光学清除则直接结束，否则获得第三次示向 u_3，并得到

$$\Omega_3 \subseteq \mathcal{W}(S_3, u_3, L).$$

于是问题重新化为标准扇形收缩结构。选择

$$S_4 = S_3 + L R_{+59^\circ} u_3,$$

则第四次示向后直径按收缩系数

$$\lambda_\delta = \frac{2\sin(2\delta)}{\sin(\pi/3 - 2\delta)} \approx 0.0823055$$

缩小，即

$$D_4 \le \lambda_\delta L \le \frac{\lambda_\delta D_2}{\sqrt{3}} \approx 8.80\ \mathrm{m}.$$

再由平面 Jung 定理，

$$R_{\mathrm{MEC}}(\Omega_4) \le \frac{D_4}{\sqrt{3}} \approx 5.08\ \mathrm{m} < 20\ \mathrm{m}.$$

因此若第四检测点仍未提前进入光学清除分支，第四次示向后也必然满足 20 m 清除条件。全过程至多使用四次示向观测，首次发现后至多只需三次追加示向。程序采用有效误差 $\delta_{\mathrm{eff}} = 1.01^\circ$ 时，将上述 59°、185.19 m、106.92 m、8.80 m 和 5.08 m 相应替换为 58.99°、187.08 m、108.00 m、8.98 m 和 5.18 m，结论仍成立。

从几何意义上看，这两个选点分别承担不同任务：S_3 取最小覆盖圆圆心，是为了让“到最远可能目标位置”的距离最小，从而保证第三次观测一定能收到信号；S_4 则从第三次中心示向 u_3 转 59°、前进 L，使视线夹角保持在 58°–60° 之间，既保证第四点仍能接收信号，又避免两条视线接近平行。实例中 $L = 46.3$ m 仅为示意；实际使用的 L 由 Ω_2 的最小覆盖圆半径决定。

2.6　圆弧接收约束与鲁棒接收域的化简

对固定 ρ，圆弧上每个可能的源位置 $G(\varphi)$ 都给出圆盘约束 $S_2 \in B(G(\varphi), \rho)$。对同时满足两端点约束的点，有 $a\cos\delta \ge |b|\sin\delta$，故 $a \ge 0$。此时 $a\cos\varphi + b\sin\varphi$ 在区间 $[-\delta, \delta]$ 内的最小值必位于一个端点，具体由 b 的符号决定，因而连续圆盘交集精确化为式 (11) 的透镜形区域。

对固定 φ，$\|S_2 - G\|^2$ 关于径向距离 t 为凸函数，在闭区间 $[0, \rho]$ 上的最大值只需检

查 $t = 0$ 与 $t = \rho$。前者给出 $B(S_1, \rho)$，后者给出上述透镜。由于两类接收约束在可行点上均随 ρ 增大而放宽，取 $\rho = \rho_-$ 即得正文式 (12)。实际定位区域还与圆域 $\mathcal{D}$ 相交，故该域保证接收，但不必包含全部可接收位置；排除 S_1 仅是为了避免原地重复测向。

附录 C　源程序代码

3.1　数据结构与问题一的求解程序

`models.py` 定义点、示向观测、目标状态与最小覆盖圆等在各问之间共用的数据结构。`geometry.py` 是几何内核，其中 `clip_bearing_wedge` 把一次示向化为半平面裁剪，`polygon_diameter` 用旋转卡壳求凸多边形直径，`minimum_enclosing_circle` 求最小覆盖圆，`diameter_circle_covers` 按式 (8) 判定以直径端点为直径的圆能否覆盖整个定位域。`localization.py` 中的 `BearingLocalizer` 依次累积示向观测并维护相容定位域，`q1_report` 输出该域的顶点、直径与覆盖结论，两者共同构成问题一的完整求解程序。

```python
from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum


@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float


@dataclass(frozen=True, slots=True)
class Observation:
    position: Point
    bearing_deg: float


class TargetStatus(str, Enum):
    UNKNOWN = "unknown"
    DETECTED = "detected"
    LOCALIZING = "localizing"
    CLEARABLE = "clearable"
    REMOVED = "removed"


@dataclass(slots=True)
class TargetState:
    channel: int
    polygon: list[Point]
    # Convex constraints from the arena, received bearings and positive
    # reception disks. polygon is a conservative convex hull after applying
    # the non-convex no-signal exclusions kept below.
    positive_polygon: list[Point] = field(default_factory=list)
    exclusion_centers: list[Point] = field(default_factory=list)
    exclusion_resolution_m: float = 25.0
    # Every valid bearing clips the feasible region. Only this separate list
    # advances the certified Q2/Q3 observation chain.
    observations: list[Observation] = field(default_factory=list)
    guarantee_observations: list[Observation] = field(default_factory=list)
```

```
    status: TargetStatus = TargetStatus.UNKNOWN
    last_measure_result: str | None = None


@dataclass(frozen=True, slots=True)
class Circle:
    center: Point
    radius: float
```

```
from __future__ import annotations

import math
import random
from collections.abc import Callable, Iterable

from .models import Circle, Point

EPS = 1e-9


def add(a: Point, b: Point) -> Point:
    return Point(a.x + b.x, a.y + b.y)


def sub(a: Point, b: Point) -> Point:
    return Point(a.x - b.x, a.y - b.y)


def scale(a: Point, k: float) -> Point:
    return Point(a.x * k, a.y * k)


def dot(a: Point, b: Point) -> float:
    return a.x * b.x + a.y * b.y


def cross(a: Point, b: Point) -> float:
    return a.x * b.y - a.y * b.x


def distance2(a: Point, b: Point) -> float:
    dx, dy = a.x - b.x, a.y - b.y
    return dx * dx + dy * dy


def distance(a: Point, b: Point) -> float:
    return math.sqrt(distance2(a, b))


def regular_outer_polygon(center: Point, radius: float, sides: int = 128) -> list[Point]:
    """A circumscribed regular polygon, so the physical disk is never excluded."""
    if sides < 4:
        raise ValueError("sides must be at least 4")
    outer_r = radius / math.cos(math.pi / sides)
    return [
        Point(center.x + outer_r * math.cos(2 * math.pi * i / sides + math.pi / sides),
              center.y + outer_r * math.sin(2 * math.pi * i / sides + math.pi / sides))
        for i in range(sides)
    ]


def _deduplicate_polygon(points: list[Point], eps: float = 1e-7) -> list[Point]:
    if not points:
        return []
```

```
    result: list[Point] = []
    for p in points:
        if not result or distance2(p, result[-1]) > eps * eps:
            result.append(p)
    if len(result) > 1 and distance2(result[0], result[-1]) <= eps * eps:
        result.pop()
    if len(result) < 3:
        return result
    cleaned: list[Point] = []
    for i, p in enumerate(result):
        prev, nxt = result[i - 1], result[(i + 1) % len(result)]
        if abs(cross(sub(p, prev), sub(nxt, p))) > eps:
            cleaned.append(p)
    return cleaned or result


def clip_halfplane(
    polygon: list[Point], signed_value: Callable[[Point], float], eps: float = EPS
) -> list[Point]:
    """Sutherland-Hodgman clipping, keeping signed_value(point) >= 0."""
    if not polygon:
        return []
    output: list[Point] = []
    a = polygon[-1]
    fa = signed_value(a)
    for b in polygon:
        fb = signed_value(b)
        a_in, b_in = fa >= -eps, fb >= -eps
        if a_in != b_in:
            denom = fa - fb
            if abs(denom) > eps:
                t = fa / denom
                output.append(Point(a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)))
        if b_in:
            output.append(b)
        a, fa = b, fb
    return _deduplicate_polygon(output)


def clip_bearing_wedge(
    polygon: list[Point], station: Point, bearing_deg: float, error_deg: float = 1.0
) -> list[Point]:
    lo = math.radians((bearing_deg - error_deg) % 360.0)
    hi = math.radians((bearing_deg + error_deg) % 360.0)
    u_lo = Point(math.cos(lo), math.sin(lo))
    u_hi = Point(math.cos(hi), math.sin(hi))
    result = clip_halfplane(polygon, lambda p: cross(u_lo, sub(p, station)))
    return clip_halfplane(result, lambda p: -cross(u_hi, sub(p, station)))


def clip_to_outer_disk(
    polygon: list[Point], center: Point, radius: float, sides: int = 64
) -> list[Point]:
    """Conservatively clip to a circumscribed polygon around a known disk."""
    result = polygon
    apothem = radius
    for i in range(sides):
        angle = 2 * math.pi * i / sides
        normal = Point(math.cos(angle), math.sin(angle))
        result = clip_halfplane(
            result, lambda p, n=normal: apothem - dot(sub(p, center), n)
        )
        if not result:
            break
```

```
    return result


def convex_hull(points: Iterable[Point]) -> list[Point]:
    """Monotone-chain convex hull in counter-clockwise order."""
    unique = sorted({(p.x, p.y) for p in points})
    pts = [Point(x, y) for x, y in unique]
    if len(pts) <= 1:
        return pts

    def build(sequence: list[Point]) -> list[Point]:
        half: list[Point] = []
        for point in sequence:
            while len(half) >= 2 and cross(
                sub(half[-1], half[-2]), sub(point, half[-1])
            ) <= EPS:
                half.pop()
            half.append(point)
        return half

    lower = build(pts)
    upper = build(list(reversed(pts)))
    return lower[:-1] + upper[:-1]


def exclusion_aware_convex_outer(
    positive_polygon: list[Point],
    exclusion_centers: Iterable[Point],
    exclusion_radius: float = 1000.0,
    resolution: float = 25.0,
    max_depth: int = 10,
) -> list[Point]:
    """Certified outer convex hull after subtracting no-signal disks.

    A quadtree cell is discarded only when its entire clipped rectangle lies
    inside an exclusion disk. Boundary cells are retained, so the returned
    hull may be conservative but can never exclude a feasible target.
    """
    centers = list(exclusion_centers)
    if not positive_polygon or not centers:
        return list(positive_polygon)

    min_x = min(p.x for p in positive_polygon)
    max_x = max(p.x for p in positive_polygon)
    min_y = min(p.y for p in positive_polygon)
    max_y = max(p.y for p in positive_polygon)
    kept: list[Point] = []

    def clip_box(x0: float, x1: float, y0: float, y1: float) -> list[Point]:
        clipped = clip_halfplane(positive_polygon, lambda p: p.x - x0)
        clipped = clip_halfplane(clipped, lambda p: x1 - p.x)
        clipped = clip_halfplane(clipped, lambda p: p.y - y0)
        return clip_halfplane(clipped, lambda p: y1 - p.y)

    def min_distance_to_box(center: Point, x0: float, x1: float, y0: float, y1: float) -> float:
        dx = max(x0 - center.x, 0.0, center.x - x1)
        dy = max(y0 - center.y, 0.0, center.y - y1)
        return math.hypot(dx, dy)

    def visit(x0: float, x1: float, y0: float, y1: float, depth: int) -> None:
        clipped = clip_box(x0, x1, y0, y1)
        if not clipped:
            return
        # A convex function reaches its maximum over a polygon at a vertex.
```

```
        if any(
            max(distance(center, point) for point in clipped)
            <= exclusion_radius + EPS
            for center in centers
        ):
            return
        touches_exclusion = any(
            min_distance_to_box(center, x0, x1, y0, y1)
            < exclusion_radius
            for center in centers
        )
        if (
            not touches_exclusion
            or depth >= max_depth
            or max(x1 - x0, y1 - y0) <= resolution
        ):
            kept.extend(clipped)
            return
        xm = (x0 + x1) / 2
        ym = (y0 + y1) / 2
        visit(x0, xm, y0, ym, depth + 1)
        visit(xm, x1, y0, ym, depth + 1)
        visit(x0, xm, ym, y1, depth + 1)
        visit(xm, x1, ym, y1, depth + 1)

    visit(min_x, max_x, min_y, max_y, 0)
    return convex_hull(kept)


def polygon_diameter(polygon: list[Point]) -> tuple[float, list[tuple[Point, Point]]]:
    """Rotating-calipers diameter for a counter-clockwise convex polygon."""
    n = len(polygon)
    if n < 2:
        return 0.0, []
    if n == 2:
        return distance(polygon[0], polygon[1]), [(polygon[0], polygon[1])]
    best2 = -1.0
    pairs: list[tuple[Point, Point]] = []

    def consider(a: Point, b: Point) -> None:
        nonlocal best2, pairs
        d2 = distance2(a, b)
        tol = 1e-8 * max(1.0, best2, d2)
        if d2 > best2 + tol:
            best2, pairs = d2, [(a, b)]
        elif abs(d2 - best2) <= tol and all({a, b} != {x, y} for x, y in pairs):
            pairs.append((a, b))

    j = 1
    for i in range(n):
        ni = (i + 1) % n
        area = lambda k: abs(cross(sub(polygon[ni], polygon[i]), sub(polygon[k], polygon[i])))
        while area((j + 1) % n) > area(j) + EPS:
            j = (j + 1) % n
        consider(polygon[i], polygon[j])
        consider(polygon[ni], polygon[j])
        if abs(area((j + 1) % n) - area(j)) <= EPS:
            consider(polygon[i], polygon[(j + 1) % n])
            consider(polygon[ni], polygon[(j + 1) % n])
    return math.sqrt(max(0.0, best2)), pairs


def _contains(circle: Circle, p: Point, eps: float = 1e-7) -> bool:
    return distance2(circle.center, p) <= (circle.radius + eps) ** 2
```

```python


def _circle2(a: Point, b: Point) -> Circle:
    center = Point((a.x + b.x) / 2, (a.y + b.y) / 2)
    return Circle(center, distance(a, b) / 2)


def _circle3(a: Point, b: Point, c: Point) -> Circle | None:
    d = 2 * cross(sub(b, a), sub(c, a))
    if abs(d) <= EPS:
        return None
    a2, b2, c2 = dot(a, a), dot(b, b), dot(c, c)
    ux = (a2 * (b.y - c.y) + b2 * (c.y - a.y) + c2 * (a.y - b.y)) / d
    uy = (a2 * (c.x - b.x) + b2 * (a.x - c.x) + c2 * (b.x - a.x)) / d
    center = Point(ux, uy)
    return Circle(center, distance(center, a))


def minimum_enclosing_circle(points: Iterable[Point]) -> Circle:
    pts = list(points)
    if not pts:
        raise ValueError("minimum enclosing circle requires at least one point")
    random.Random(2026).shuffle(pts)
    circle = Circle(pts[0], 0.0)
    for i, p in enumerate(pts):
        if _contains(circle, p):
            continue
        circle = Circle(p, 0.0)
        for j in range(i):
            q = pts[j]
            if _contains(circle, q):
                continue
            circle = _circle2(p, q)
            for k in range(j):
                r = pts[k]
                if _contains(circle, r):
                    continue
                candidate = _circle3(p, q, r)
                if candidate is not None:
                    circle = candidate
                else:
                    choices = (_circle2(p, q), _circle2(p, r), _circle2(q, r))
                    circle = min(
                        (c0 for c0 in choices if all(_contains(c0, x) for x in (p, q, r))),
                        key=lambda c0: c0.radius,
                    )
    return circle


def diameter_circle_covers(polygon: list[Point]) -> tuple[bool, Circle | None]:
    diameter, pairs = polygon_diameter(polygon)
    for a, b in pairs:
        circle = _circle2(a, b)
        if all(_contains(circle, p) for p in polygon):
            return True, circle
    return False, None if not pairs else _circle2(*pairs[0])


def project_to_convex_polygon(point: Point, polygon: list[Point]) -> Point:
    """Return point itself when inside, otherwise its nearest boundary projection."""
    if not polygon:
        raise ValueError("cannot project onto an empty polygon")
    if len(polygon) == 1:
        return polygon[0]
```

```
    # The localization polygons are counter-clockwise.
    if all(cross(sub(polygon[(i + 1) % len(polygon)], polygon[i]), sub(point, polygon[i])) >= -EPS
           for i in range(len(polygon))):
        return point
    best, best_d2 = polygon[0], float("inf")
    edge_count = len(polygon) if len(polygon) > 2 else 1
    for i in range(edge_count):
        a, b = polygon[i], polygon[(i + 1) % len(polygon)]
        ab = sub(b, a)
        denom = dot(ab, ab)
        t = 0.0 if denom <= EPS else max(0.0, min(1.0, dot(sub(point, a), ab) / denom))
        candidate = add(a, scale(ab, t))
        d2 = distance2(point, candidate)
        if d2 < best_d2:
            best, best_d2 = candidate, d2
    return best
```

```
from __future__ import annotations

import math

from .geometry import (
    clip_bearing_wedge,
    clip_to_outer_disk,
    diameter_circle_covers,
    exclusion_aware_convex_outer,
    minimum_enclosing_circle,
    polygon_diameter,
    regular_outer_polygon,
)
from .models import Circle, Observation, Point, TargetState, TargetStatus


class InconsistentObservations(RuntimeError):
    pass


class BearingLocalizer:
    """Q1 incremental convex feasible-region estimator."""

    def __init__(
        self,
        arena_radius: float = 1800.0,
        bearing_error_deg: float = 1.01,
        max_reception_radius: float = 1500.0,
        polygon_sides: int = 128,
    ) -> None:
        self.arena_radius = arena_radius
        self.bearing_error_deg = bearing_error_deg
        self.max_reception_radius = max_reception_radius
        self.polygon_sides = polygon_sides

    def new_target(self, channel: int) -> TargetState:
        initial = regular_outer_polygon(
            Point(0.0, 0.0), self.arena_radius, self.polygon_sides
        )
        return TargetState(
            channel=channel,
            polygon=list(initial),
            positive_polygon=list(initial),
        )

    def _planning_outer(
```

```
        self, positive_polygon: list[Point], exclusion_centers: list[Point]
    ) -> tuple[list[Point], float]:
        """Rebuild with progressively finer boundary cells near clearability."""
        polygon = exclusion_aware_convex_outer(
            positive_polygon,
            exclusion_centers,
            resolution=25.0,
            max_depth=10,
        )
        if not polygon or not exclusion_centers:
            return polygon, 25.0

        resolution = 25.0
        # Each finer pass starts from the raw positive polygon and all original
        # exclusion disks, so coarse approximations never become constraints.
        radius = minimum_enclosing_circle(polygon).radius
        if radius <= 120.0:
            polygon = exclusion_aware_convex_outer(
                positive_polygon,
                exclusion_centers,
                resolution=10.0,
                max_depth=11,
            )
            resolution = 10.0
            radius = minimum_enclosing_circle(polygon).radius
        if radius <= 55.0:
            polygon = exclusion_aware_convex_outer(
                positive_polygon,
                exclusion_centers,
                resolution=2.0,
                max_depth=13,
            )
            resolution = 2.0
            radius = minimum_enclosing_circle(polygon).radius
        if radius <= 28.0:
            polygon = exclusion_aware_convex_outer(
                positive_polygon,
                exclusion_centers,
                resolution=0.5,
                max_depth=15,
            )
            resolution = 0.5
        return polygon, resolution

    def add_direction(self, target: TargetState, position: Point, bearing_deg: float) -> None:
        base = target.positive_polygon or target.polygon
        polygon = clip_bearing_wedge(
            base, position, bearing_deg, self.bearing_error_deg
        )
        polygon = clip_to_outer_disk(
            polygon, position, self.max_reception_radius, max(32, self.polygon_sides // 2)
        )
        if not polygon:
            raise InconsistentObservations(
                f"channel {target.channel}: bearing observations have empty intersection"
            )
        plan_poly, resolution = self._planning_outer(
            polygon, target.exclusion_centers
        )
        if not plan_poly:
            raise InconsistentObservations(
                f"channel {target.channel}: exclusions leave no feasible region"
            )
        target.positive_polygon = polygon
```

```
        target.polygon = plan_poly
        target.exclusion_resolution_m = resolution
        target.observations.append(Observation(position, bearing_deg % 360.0))
        target.status = TargetStatus.LOCALIZING
        if self.is_clearable(target):
            target.status = TargetStatus.CLEARABLE

    def add_no_signal(
        self, target: TargetState, position: Point, exclusion_radius: float = 1000.0
    ) -> None:
        """Keep a raw exclusion constraint and rebuild a safe planning hull."""
        appended = all(
            math.dist((position.x, position.y), (old.x, old.y)) > 1e-7
            for old in target.exclusion_centers
        )
        if appended:
            target.exclusion_centers.append(position)
        positive = target.positive_polygon or target.polygon
        if abs(exclusion_radius - 1000.0) <= 1e-9:
            polygon, resolution = self._planning_outer(
                positive, target.exclusion_centers
            )
        else:
            polygon = exclusion_aware_convex_outer(
                positive,
                target.exclusion_centers,
                exclusion_radius=exclusion_radius,
                resolution=2.0,
                max_depth=13,
            )
            resolution = 2.0
        if not polygon:
            if appended:
                target.exclusion_centers.pop()
            raise InconsistentObservations(
                f"channel {target.channel}: no-signal exclusions leave no feasible region"
            )
        target.polygon = polygon
        target.exclusion_resolution_m = resolution

    @staticmethod
    def diameter(target: TargetState) -> float:
        return polygon_diameter(target.polygon)[0]

    @staticmethod
    def covering_circle(target: TargetState) -> Circle:
        return minimum_enclosing_circle(target.polygon)

    def is_clearable(self, target: TargetState, clear_radius: float = 20.0) -> bool:
        return bool(target.polygon) and self.covering_circle(target).radius <= clear_radius + 1e
            -6

    @staticmethod
    def q1_report(target: TargetState) -> dict[str, object]:
        diameter, pairs = polygon_diameter(target.polygon)
        covered, diameter_circle = diameter_circle_covers(target.polygon)
        mec = minimum_enclosing_circle(target.polygon)
        return {
            "vertex_count": len(target.polygon),
            "diameter": diameter,
            "diameter_pair_count": len(pairs),
            "diameter_circle_covers": covered,
            "diameter_circle": diameter_circle,
            "minimum_enclosing_circle": mec,
```

```
        }
```

3.2 问题二的求解程序

active_sensing.py 对应问题二：q2_analytic_candidates 给出式 (14) 的两个解析基准点，in_q2_robust_region 按式 (12) 核对候选点是否落在鲁棒接收域内，ActiveSensor 在候选点上预测下一次示向并调用问题一的裁剪评估器比较裁剪后的直径，取直径较小的一侧作为第二次检测点，此后在偶数次观测后改取定位域的最小覆盖圆圆心，为第三次示向提供保证接收的驻留点。

```python
from __future__ import annotations

import math

from .geometry import clip_bearing_wedge, distance, minimum_enclosing_circle, polygon_diameter
from .models import Point, TargetState


def rotated_step(origin: Point, bearing_deg: float, offset_deg: float, length: float) -> Point:
    angle = math.radians(bearing_deg + offset_deg)
    return Point(origin.x + length * math.cos(angle), origin.y + length * math.sin(angle))


def q2_analytic_candidates(
    first_position: Point,
    first_bearing_deg: float,
    min_reception_radius: float = 1000.0,
    error_deg: float = 1.0,
) -> tuple[Point, Point]:
    """Robust candidates S1 + rho_min R(+/-(60-error)) u1."""
    gamma_deg = 60.0 - error_deg
    return (
        rotated_step(first_position, first_bearing_deg, gamma_deg, min_reception_radius),
        rotated_step(first_position, first_bearing_deg, -gamma_deg, min_reception_radius),
    )


def in_q2_robust_region(a: float, b: float, rho_min: float = 1000.0, delta_deg: float = 1.0) ->
     bool:
    delta = math.radians(delta_deg)
    r2 = a * a + b * b
    return (
        r2 <= rho_min * rho_min + 1e-7
        and r2 <= 2 * rho_min * (a * math.cos(delta) - abs(b) * math.sin(delta)) + 1e-7
    )


class ActiveSensor:
    """Q2 candidate generator with a Q1-based worst-case geometric scorer."""

    def __init__(self, min_reception_radius: float = 1000.0, error_deg: float = 1.01) -> None:
        self.min_reception_radius = min_reception_radius
        self.error_deg = error_deg

    def choose_next(self, target: TargetState, current: Point) -> Point:
        if not target.observations:
            raise ValueError("target has no direction observation")
        first = target.observations[0]
        if len(target.observations) == 1:
```

```
            candidates = q2_analytic_candidates(
                first.position,
                first.bearing_deg,
                self.min_reception_radius,
                self.error_deg,
            )
            return min(candidates, key=lambda p: distance(current, p))

        mec = minimum_enclosing_circle(target.polygon)
        if len(target.observations) % 2 == 0 and distance(current, mec.center) > 5.0:
            return mec.center

        latest = target.observations[-1]
        diameter = polygon_diameter(target.polygon)[0]
        step = min(self.min_reception_radius, max(40.0, diameter))
        offset_deg = 60.0 - self.error_deg
        candidates = (
            rotated_step(latest.position, latest.bearing_deg, offset_deg, step),
            rotated_step(latest.position, latest.bearing_deg, -offset_deg, step),
        )
        # Predict each candidate's bearing toward the current feasible-region center,
        # then use the exact Q1 clipping evaluator and prefer the smaller diameter.
        def score(p: Point) -> tuple[float, float]:
            angle = math.degrees(math.atan2(mec.center.y - p.y, mec.center.x - p.x)) % 360.0
            predicted = clip_bearing_wedge(target.polygon, p, angle, self.error_deg)
            d = polygon_diameter(predicted)[0] if predicted else float("inf")
            return d, distance(current, p)

        return min(candidates, key=score)
```

3.3 问题三的求解程序

q3_path_planner.py 的第 1–149 行给出式 (18) 的七点兜底覆盖布局、路线长度评价、插入式选点与覆盖状态管理。baseline/controller.py 分两段摘录：第 91–131 行是滚动主循环与任务分发，第 222–411 行是示向累积、四观测保证链、反向约束处理与下一检测点选择。optimized/controller.py 是加入零绕行机会观测后的完整控制器，它只在机器狗必须经过的位置增加示向，因而不增加移动距离。

```
"""Problem 3 rolling path planner, independent of simulator communication."""

from __future__ import annotations

from dataclasses import dataclass, field
from math import cos, pi, sin
from collections.abc import Iterable, Sequence

from .geometry import distance
from .models import Point

ARENA_RADIUS = 1800.0
GUARANTEE_RADIUS = 1000.0
DEFAULT_COVER_RING_RADIUS = 1130.0


@dataclass(frozen=True, slots=True)
class Task:
    task_id: str
    point: Point
    kind: str = "cover"
```

```
    channel: int | None = None
    service_time_s: float = 0.0
    priority: float = 0.0
    mandatory: bool = True


def route_length(start: Point, route: Sequence[Task]) -> float:
    total = 0.0
    current = start
    for task in route:
        total += distance(current, task.point)
        current = task.point
    return total


def choose_route_aware_candidate(
    current: Point, candidates: Sequence[Point], continuation_anchors: Sequence[Point] = ()
) -> Point:
    if not candidates:
        raise ValueError("at least one candidate is required")

    def score(point: Point) -> float:
        continuation = min(
            (distance(point, anchor) for anchor in continuation_anchors), default=0.0
        )
        return distance(current, point) + continuation

    return min(candidates, key=score)


def cheapest_insertion_index(start: Point, route: Sequence[Task], task: Task) -> tuple[int,
     float]:
    best_index, best_delta = 0, float("inf")
    for index in range(len(route) + 1):
        before = start if index == 0 else route[index - 1].point
        if index == len(route):
            delta = distance(before, task.point)
        else:
            after = route[index].point
            delta = distance(before, task.point) + distance(task.point, after) - distance(before,
                 after)
        if delta < best_delta:
            best_index, best_delta = index, delta
    return best_index, best_delta


def nearest_neighbor_route(start: Point, tasks: Iterable[Task]) -> list[Task]:
    remaining = list(tasks)
    route: list[Task] = []
    current = start
    while remaining:
        index = min(range(len(remaining)), key=lambda i: distance(current, remaining[i].point))
        task = remaining.pop(index)
        route.append(task)
        current = task.point
    return route


def two_opt_open(start: Point, route: Sequence[Task]) -> list[Task]:
    best = list(route)
    if len(best) < 3:
        return best
    while True:
        best_length = route_length(start, best)
```

```
        improved = False
        for i in range(len(best) - 1):
            for j in range(i + 1, len(best)):
                candidate = best[:i] + list(reversed(best[i : j + 1])) + best[j + 1 :]
                if route_length(start, candidate) + 1e-9 < best_length:
                    best = candidate
                    improved = True
                    break
            if improved:
                break
        if not improved:
            return best


def seven_point_cover(radius: float = DEFAULT_COVER_RING_RADIUS) -> list[Task]:
    tasks = [Task("cover-center", Point(0.0, 0.0), service_time_s=119.0)]
    for k in range(6):
        angle = k * pi / 3.0
        tasks.append(
            Task(
                f"cover-{k}",
                Point(radius * cos(angle), radius * sin(angle)),
                service_time_s=120.0,
            )
        )
    return tasks


@dataclass(slots=True)
class ChannelCoverage:
    """Per-channel no-signal history and strict seven-point certification."""

    channel: int
    status: str = "unknown"
    no_signal_points: list[Point] = field(default_factory=list)
    fallback_scanned: set[str] = field(default_factory=set)

    def record(self, point: Point, result: str, fallback_id: str | None = None) -> None:
        if result == "no_signal":
            if all(distance(point, old) > 1e-7 for old in self.no_signal_points):
                self.no_signal_points.append(point)
            if fallback_id is not None:
                self.fallback_scanned.add(fallback_id)
            if len(self.fallback_scanned) == 7:
                self.status = "absent"
        elif result == "direction":
            self.status = "localizing"
        elif result == "near":
            self.status = "clearable"
        else:
            raise ValueError(f"unknown measure result: {result}")

    def mark_cleared(self) -> None:
        self.status = "cleared"

    @property
    def needs_search(self) -> bool:
        return self.status == "unknown"

    @property
    def certified_complete(self) -> bool:
        return self.status in {"absent", "cleared"}

    def history_key(self, digits: int = 3) -> tuple[tuple[float, float], ...]:
```

```
        return tuple((round(p.x, digits), round(p.y, digits)) for p in self.no_signal_points)
```

```
    def _run_q3_rolling(self) -> None:
        planner = RollingPlanner()
        self.q3_planner = planner
        self.q3_coverage = CoverageManager()
        for task in seven_point_cover():
            planner.upsert(task)
        while True:
            task = planner.next_task(self.client.position)
            if task is None:
                if self.q3_coverage.all_channels_certified():
                    break
                self._restore_missing_q3_tasks()
                task = planner.next_task(self.client.position)
                if task is None:
                    raise RuntimeError(
                        "Q3 task queue is empty before every channel is cleared or absent"
                    )
            self._check_deadline()
            planner.remove(task.task_id)
            if task.kind != "cover" and any(item.kind == "cover" for item in planner.tasks.values
                ()):
                self.diag_.begin_non_cover_excursion(self.client.position)
            if task.kind == "cover":
                self._scan_q3_search_point(task.point, fallback_id=task.task_id, force_fallback=
                    True)
                self.diag_.end_at_cover(task.point)
            elif task.kind == "localize":
                response = self._measure(task.point, int(task.channel), role="localize")
                self._consume_measure(int(task.channel), task.point, response, defer_near_clear=
                    True)
                self._piggyback_q3_search(task.point)
                if response["measure_result"] == "no_signal":
                    # This should not occur at the first Q2 robust point for an omnidirectional
                    # source. Keep the target alive so later coverage observations can repair it.
                    self.logger.warning("channel %s: no signal at planned Q3 localization point",
                        task.channel)
            elif task.kind == "clear":
                self._execute_planned_clear(task)
                self._piggyback_q3_search(task.point)
            else:
                raise RuntimeError(f"unknown Q3 task kind: {task.kind}")

        if not self.q3_coverage.all_channels_certified():
            raise RuntimeError("Q3 attempted to exit with uncertified channels")
```

```
    def _consume_measure(
        self,
        channel: int,
        point: Point,
        response: dict[str, object],
        defer_near_clear: bool = False,
        advances_guarantee: bool = True,
    ) -> None:
        result = str(response["measure_result"])
        target = self.targets.get(channel)
        if result == "no_signal":
            if target is not None:
                target.last_measure_result = result
                try:
                    self.localizer.add_no_signal(target, point)
                except InconsistentObservations:
                    # A planned robust point returning no signal contradicts
```

```
                # the ideal model. Preserve the previous safe hull.
                self.logger.exception(
                    "channel %d: inconsistent no-signal exclusion", channel
                )
            if defer_near_clear and target.observations and self.q3_planner:
                # Defer regeneration until another coverage observation changes the state;
                # regenerating the same deterministic point here would loop forever.
                cover_left = any(t.kind == "cover" for t in self.q3_planner.tasks.values())
                if not cover_left:
                    if len(target.guarantee_observations) == 1:
                        first = target.guarantee_observations[0]
                        alternatives = q2_analytic_candidates(
                            first.position,
                            first.bearing_deg,
                            error_deg=self.active_sensor.error_deg,
                        )
                        fallback = max(
                            alternatives,
                            key=lambda p: ((p.x - self.client.position.x) ** 2 + (p.y - self.
                                client.position.y) ** 2),
                        )
                    else:
                        fallback = self.active_sensor.choose_next(target, self.client.position)
                    self.q3_planner.upsert(
                        Task(
                            f"localize-{channel}", fallback, "localize", channel, 6.0,
                            priority=0.5,
                        )
                    )
            return
        if target is None:
            target = self.localizer.new_target(channel)
            if self.q3_coverage is not None:
                # Reuse every no-signal exclusion collected before discovery.
                target.exclusion_centers.extend(
                    self.q3_coverage.channels[channel].no_signal_points
                )
            target.status = TargetStatus.DETECTED
            self.targets[channel] = target
        target.last_measure_result = result
        if result == "near":
            self._needs_new_obs.discard(channel)
            if defer_near_clear and self.q3_planner:
                target.status = TargetStatus.CLEARABLE
                self.q3_planner.remove(f"localize-{channel}")
                self.q3_planner.upsert(
                    Task(f"clear-{channel}", point, "clear", channel, 5.0, priority=4.0)
                )
                return
            clear = self._clear(point, channel, role="clear")
            if clear["clear_result"] == "success":
                target.status = TargetStatus.REMOVED
                self.removed.add(channel)
            return
        is_new = not target.observations
        accepted_dir = False
        try:
            self.localizer.add_direction(target, point, float(response["svd_deg"]))
        except InconsistentObservations:
            self.logger.exception("inconsistent direction for channel %d", channel)
            if all(distance(point, old) > 1e-7 for old in self._bad_meas_pts[channel]):
                self._bad_meas_pts[channel].append(point)
            # A first bearing can only conflict with inherited no-signal
            # exclusions or numerical approximations. Quarantine those
```

```
            # exclusions and retry from the arena rather than leaving a target
            # with no observations.
            if is_new and target.exclusion_centers:
                inherited = list(target.exclusion_centers)
                target.exclusion_centers.clear()
                try:
                    self.localizer.add_direction(
                        target, point, float(response["svd_deg"])
                    )
                except InconsistentObservations:
                    target.exclusion_centers.extend(inherited)
                else:
                    self._held_exclusions[channel].extend(inherited)
                    self.logger.warning(
                        "channel %d: quarantined %d inherited no-signal exclusions",
                        channel,
                        len(inherited),
                    )
                    accepted_dir = True
            if is_new and not accepted_dir:
                self.targets.pop(channel, None)
                raise RuntimeError(
                    f"channel {channel}: first direction is inconsistent even "
                    "after removing inherited exclusions"
                )
        else:
            accepted_dir = True

        if accepted_dir:
            observation = target.observations[-1]
            # An arbitrary first bearing is a valid S1 even when discovered at
            # an opportunity stop. Later opportunity bearings update geometry
            # but cannot silently advance the certified construction.
            # The Q2 certificate contains at most four deliberately constructed
            # bearings. Recovery/task observations taken after that point still
            # refine the full feasible set, but must not silently extend the
            # four-observation certificate.
            if (
                not target.guarantee_observations
                or advances_guarantee
                and len(target.guarantee_observations) < 4
            ):
                target.guarantee_observations.append(observation)
            self._needs_new_obs.discard(channel)
        if defer_near_clear:
            self._sync_q3_target_task(target)

    def _sync_q3_target_task(self, target: TargetState) -> None:
        planner = self.q3_planner
        if planner is None or target.channel in self.removed:
            return
        planner.remove(f"localize-{target.channel}")
        planner.remove(f"clear-{target.channel}")
        if (
            self.localizer.is_clearable(target)
            and target.channel not in self._needs_new_obs
        ):
            center = self.localizer.covering_circle(target).center
            target.status = TargetStatus.CLEARABLE
            planner.upsert(
                Task(f"clear-{target.channel}", center, "clear", target.channel, 5.0, priority
                    =3.0)
            )
            return
```

```
        next_point = self._q3_next_detection_point(target)
        planner.upsert(
            Task(f"localize-{target.channel}", next_point, "localize", target.channel, 6.0,
                priority=1.0)
        )

    def _q3_next_detection_point(self, target: TargetState) -> Point:
        planner = self.q3_planner
        assert planner is not None
        anchors = planner.continuation_anchors(self.client.position, f"localize-{target.channel}"
            )
        guarantee = target.guarantee_observations
        if len(guarantee) == 1:
            first = guarantee[0]
            candidates = self._unblocked_candidates(
                target.channel,
                q2_analytic_candidates(
                    first.position,
                    first.bearing_deg,
                    error_deg=self.active_sensor.error_deg,
                ),
            )
            return choose_route_aware_candidate(self.client.position, candidates, anchors)
        if len(guarantee) == 2:
            desired = min(anchors, key=lambda p: self._point_distance_to_polygon(p, target.
                polygon), default=None)
            if desired is None:
                desired = self.localizer.covering_circle(target).center
            projected = project_to_convex_polygon(desired, target.polygon)
            choices = self._unblocked_candidates(
                target.channel,
                (projected, self.localizer.covering_circle(target).center, *target.polygon),
            )
            return min(choices, key=lambda p: distance(self.client.position, p))
        if len(guarantee) == 3:
            latest = guarantee[-1]
            length = max(40.0, min(1000.0, polygon_diameter(target.polygon)[0]))
            offset_deg = 60.0 - self.active_sensor.error_deg
            candidates = (
                rotated_step(latest.position, latest.bearing_deg, offset_deg, length),
                rotated_step(latest.position, latest.bearing_deg, -offset_deg, length),
            )
            candidates = self._unblocked_candidates(target.channel, candidates)
            return choose_route_aware_candidate(self.client.position, candidates, anchors)
        proposed = self.active_sensor.choose_next(target, self.client.position)
        choices = self._unblocked_candidates(
            target.channel,
            (proposed, self.localizer.covering_circle(target).center, *target.polygon),
        )
        return min(choices, key=lambda p: distance(self.client.position, p))
```

```
"""Problem 3 anytime controller with two explicitly separated observation roles.

Task observations preserve the Q2/Q3 guarantee chain. Opportunistic observations
are allowed only at a point the robot must already visit, so they add no movement.
They are budgeted and filtered to prevent uncontrolled channel scanning.
"""

from __future__ import annotations

import math
from collections import defaultdict

from baseline.controller import RobotController
```

```python
from common.geometry import distance, minimum_enclosing_circle
from common.models import Point, TargetStatus
from common.q3_path_planner import CoverageManager, RollingPlanner, seven_point_cover


class AnytimeRobotController(RobotController):
    """A platform-ready Q3 controller using task and opportunistic observations."""

    def __init__(
        self,
        client,
        safety_margin_s: float = 15.0,
        max_opportunistic_per_stop: int = 4,
        max_unknown_per_stop: int = 2,
        min_unknown_gain: float = 0.12,
        min_angular_information: float = 0.35,
        no_signal_cooldown_stops: int = 2,
    ) -> None:
        super().__init__(client, mode="q3", safety_margin_s=safety_margin_s)
        self.opp_cap = max_opportunistic_per_stop
        self.unk_cap = max_unknown_per_stop
        self.min_unknown_gain = min_unknown_gain
        self.min_angle_info = min_angular_information
        self.cooldown_stops = no_signal_cooldown_stops
        self._stop_index = 0
        self._opp_block_until: dict[int, int] = defaultdict(int)
        self._measured_points: dict[int, list[Point]] = defaultdict(list)

    def _run_q3_rolling(self) -> None:
        planner = RollingPlanner()
        self.q3_planner = planner
        self.q3_coverage = CoverageManager()
        for task in seven_point_cover():
            planner.upsert(task)

        while True:
            task = planner.next_task(self.client.position)
            if task is None:
                if self.q3_coverage.all_channels_certified():
                    break
                self._restore_missing_q3_tasks()
                task = planner.next_task(self.client.position)
                if task is None:
                    raise RuntimeError(
                        "Q3 anytime task queue is empty before every channel is cleared or absent"
                    )
            self._check_deadline()
            planner.remove(task.task_id)
            if task.kind != "cover" and any(item.kind == "cover" for item in planner.tasks.values
                ()):
                self.diag_.begin_non_cover_excursion(self.client.position)

            if task.kind == "cover":
                self._scan_task_search_point(task.point, task.task_id)
                self.diag_.end_at_cover(task.point)
            elif task.kind == "localize":
                channel = int(task.channel)
                response = self._measure(task.point, channel, role="task_localize")
                self._remember_measurement(channel, task.point)
                self._consume_measure(channel, task.point, response, defer_near_clear=True)
                self._take_opportunistic_observations(task.point, exclude={channel})
            elif task.kind == "clear":
                self._execute_planned_clear(task)
                self._take_opportunistic_observations(task.point, exclude={int(task.channel)})
```

```
            else:
                raise RuntimeError(f"unknown Q3 anytime task kind: {task.kind}")

        if not self.q3_coverage.all_channels_certified():
            raise RuntimeError("Q3 anytime attempted to exit with uncertified channels")

    def _scan_task_search_point(self, point: Point, fallback_id: str) -> None:
        """Mandatory seven-point coverage; it is never charged to opportunity budget."""
        coverage = self.q3_coverage
        assert coverage is not None
        channels = [
            channel
            for channel, state in coverage.channels.items()
            if state.needs_search and fallback_id not in state.fallback_scanned
        ]
        if self.client.current_channel in channels:
            channels.remove(self.client.current_channel)
            channels.insert(0, self.client.current_channel)
        for channel in channels:
            self._check_deadline()
            response = self._measure(point, channel, role="task_cover")
            self._remember_measurement(channel, point)
            result = str(response["measure_result"])
            coverage.channels[channel].record(point, result, fallback_id)
            self._consume_measure(channel, point, response, defer_near_clear=True)

        # Active targets can still benefit from this already mandatory stop.
        self._take_opportunistic_observations(point)

    def _take_opportunistic_observations(self, point: Point, exclude: set[int] | None = None) ->
      None:
        """Use a zero-detour stop, subject to reception, information and count budgets."""
        self._stop_index += 1
        excluded = exclude or set()
        candidates: list[tuple[float, int, str]] = []

        # First preference: improve a target already being localized. The MEC
        # containment test guarantees the whole feasible region is within 1000 m.
        for channel, target in self.targets.items():
            if channel in excluded or channel in self.removed:
                continue
            if target.status not in {TargetStatus.DETECTED, TargetStatus.LOCALIZING}:
                continue
            if self._seen_here(channel, point) or self._cooling_down(channel):
                continue
            mec = minimum_enclosing_circle(target.polygon)
            if distance(point, mec.center) + mec.radius > 1000.0 + 1e-7:
                continue
            score = self._angular_information(target, point, mec.center)
            if score >= self.min_angle_info:
                candidates.append((2.0 + score, channel, "active target"))

        # Second preference: discover an unknown channel only when this stop adds
        # substantial uncovered area. Cap this class independently.
        coverage = self.q3_coverage
        if coverage is not None:
            unknown: list[tuple[float, int, str]] = []
            for channel, state in coverage.channels.items():
                if channel in excluded or not state.needs_search:
                    continue
                if self._seen_here(channel, point) or self._cooling_down(channel):
                    continue
                new_count, uncovered_n = coverage.marginal_witness_count(channel, point)
                gain = new_count / uncovered_n if uncovered_n else 0.0
```

```
                if gain >= self.min_unknown_gain:
                    unknown.append((gain, channel, "coverage gain"))
            unknown.sort(reverse=True)
            candidates.extend(unknown[: self.unk_cap])

        candidates.sort(reverse=True)
        selected = candidates[: self.opp_cap]
        if not selected:
            return
        self.diag_.record_piggyback(len(selected))
        for _, channel, _ in selected:
            self._check_deadline()
            response = self._measure(point, channel, role="opportunistic_observation")
            self._remember_measurement(channel, point)
            result = str(response["measure_result"])
            if result == "no_signal":
                self._opp_block_until[channel] = (
                    self._stop_index + self.cooldown_stops
                )
            if coverage is not None and coverage.channels[channel].needs_search:
                # Opportunity samples help planning but do not count as one of the
                # seven mandatory absence certificates.
                coverage.channels[channel].record(point, result, None)
            self._consume_measure(
                channel,
                point,
                response,
                defer_near_clear=True,
                advances_guarantee=False,
            )

    def _remember_measurement(self, channel: int, point: Point) -> None:
        if not self._seen_here(channel, point):
            self._measured_points[channel].append(point)

    def _seen_here(self, channel: int, point: Point) -> bool:
        return any(distance(point, old) <= 1e-6 for old in self._measured_points[channel])

    def _cooling_down(self, channel: int) -> bool:
        return self._stop_index < self._opp_block_until[channel]

    @staticmethod
    def _angular_information(target, point: Point, center: Point) -> float:
        if not target.observations:
            return 1.0
        predicted = math.degrees(math.atan2(center.y - point.y, center.x - point.x)) % 180.0
        best = 0.0
        for observation in target.observations:
            old = observation.bearing_deg % 180.0
            delta = abs(predicted - old)
            separation = min(delta, 180.0 - delta)
            best = max(best, separation)
        return min(1.0, best / 90.0)
```

3.4 问题四的求解程序

`geometry.py` 实现式 (23) 与式 (24) 的三角覆盖证书，其中 25 个严格搜索点与 36 个证书三角形以常量表给出，`certifiable` 检查三角形的最长边不超过保证接收半径 1000 m，`CellEvidence` 记录各面片的证据状态并给出式 (24) 的排除判定。`planner.py`

的第 422–710 行是问题四的在线决策主体，包括 Lazy 候选打分、等长 Hamilton 路线选择、定位探针与终止判定。optimized/runner.py 的第 40–222 行给出同心环布点的 25 点坐标、证书面片生成与固定路线，是正文采用的布点方案。

```python
"""Problem 4 geometry and strict 25-point coverage certificate.

This simulator-independent module contains only mathematical primitives:
the certified triangular mesh, convex clipping, and minimum enclosing circle.
The online state and route policy are implemented in
``common.planner``.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum
from math import cos, hypot, pi, sin, sqrt
from typing import Sequence


# ---------------------------------------------------------------------------
# Model parameters and certified 25-point construction
# ---------------------------------------------------------------------------

ARENA_RADIUS = 1800.0
GUARANTEED_RANGE = 1000.0
MAX_RECEPTION_RANGE = 1500.0
BEARING_ERROR_DEG = 1.01
CLEAR_RADIUS = 20.0
EPS = 1e-9
MESH_EDGE_TOLERANCE = 1e-7
GOLDEN_ANGLE_DEG = 137.507764

TRIANGLE25_LATTICE_INDICES = (
    (-3, 1),
    (-3, 2),
    (-2, -1),
    (-2, 0),
    (-2, 1),
    (-2, 2),
    (-1, -2),
    (-1, -1),
    (-1, 0),
    (-1, 1),
    (-1, 2),
    (0, -2),
    (0, -1),
    (0, 0),
    (0, 1),
    (0, 2),
    (1, -3),
    (1, -2),
    (1, -1),
    (1, 0),
    (1, 1),
    (2, -3),
    (2, -2),
    (2, -1),
    (2, 0),
)
TRIANGLE25_FACES = (
    (9, 8, 13),
    (8, 12, 13),
```

```
    (12, 8, 7),
    (9, 10, 5),
    (1, 4, 5),
    (4, 9, 5),
    (4, 8, 9),
    (0, 4, 1),
    (2, 6, 7),
    (14, 10, 9),
    (14, 9, 13),
    (10, 14, 15),
    (14, 20, 15),
    (3, 2, 7),
    (8, 3, 7),
    (4, 3, 8),
    (0, 3, 4),
    (11, 12, 7),
    (6, 11, 7),
    (11, 17, 12),
    (17, 11, 16),
    (12, 18, 13),
    (23, 18, 22),
    (17, 18, 12),
    (18, 17, 22),
    (17, 21, 22),
    (21, 17, 16),
    (19, 14, 13),
    (19, 23, 24),
    (20, 19, 24),
    (14, 19, 20),
    (19, 18, 23),
    (18, 19, 13),
)
TRIANGLE25_ROUTE_INDICES = (
    13,
    8,
    3,
    0,
    1,
    4,
    5,
    9,
    10,
    15,
    14,
    20,
    24,
    19,
    23,
    18,
    22,
    21,
    16,
    17,
    12,
    11,
    6,
    2,
    7,
)


@dataclass(frozen=True, order=True)
class Point:
    x: float
```

```
    y: float


TRIANGLE25_POINTS = tuple(
    Point(1000.0 * i + 500.0 * j + 150.0, 500.0 * sqrt(3.0) * j + 50.0 * sqrt(3.0))
    for i, j in TRIANGLE25_LATTICE_INDICES
)
STRICT_SEARCH_ROUTE = tuple(TRIANGLE25_POINTS[i] for i in TRIANGLE25_ROUTE_INDICES)


# ---------------------------------------------------------------------------
# Basic geometry and strict-search certificate
# ---------------------------------------------------------------------------


def distance(a: Point, b: Point) -> float:
    return hypot(a.x - b.x, a.y - b.y)


def cross(a: Point, b: Point, c: Point) -> float:
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x)


@dataclass(frozen=True)
class TriangleCell:
    cell_id: str
    vertices: tuple[Point, Point, Point]

    @property
    def certificate_points(self) -> tuple[Point, ...]:
        return self.vertices

    def contains(self, point: Point) -> bool:
        a, b, c = self.vertices
        signs = (cross(a, b, point), cross(b, c, point), cross(c, a, point))
        return all(value >= -EPS for value in signs) or all(
            value <= EPS for value in signs
        )

    def certifiable(self, guaranteed_range: float = GUARANTEED_RANGE) -> bool:
        return (
            max(
                distance(a, b)
                for a, b in (
                    (self.vertices[0], self.vertices[1]),
                    (self.vertices[1], self.vertices[2]),
                    (self.vertices[2], self.vertices[0]),
                )
            )
            <= guaranteed_range + EPS
        )


def build_triangle_cells(
    arena_radius: float = ARENA_RADIUS,
    guaranteed_range: float = GUARANTEED_RANGE,
) -> list[TriangleCell]:
    if (
        abs(arena_radius - ARENA_RADIUS) > EPS
        or abs(guaranteed_range - GUARANTEED_RANGE) > EPS
    ):
        raise ValueError("the certified 25-point mesh is fixed for radii 1800/1000")
    cells = [
        TriangleCell(f"t{index:02d}", tuple(TRIANGLE25_POINTS[i] for i in face))
```

```
        for index, face in enumerate(TRIANGLE25_FACES)
    ]
    if not all(cell.certifiable(guaranteed_range) for cell in cells):
        raise RuntimeError("invalid triangular certificate")
    return cells


class CellStatus(str, Enum):
    UNVISITED = "unvisited"
    PARTIAL = "partial"
    EXCLUDED = "excluded"
    DETECTED = "detected"


@dataclass
class CellEvidence:
    cell: TriangleCell
    no_signal_points: set[Point] = field(default_factory=set)
    detected_at: Point | None = None

    @property
    def status(self) -> CellStatus:
        if self.detected_at is not None:
            return CellStatus.DETECTED
        if all(
            point in self.no_signal_points for point in self.cell.certificate_points
        ):
            return CellStatus.EXCLUDED
        if self.no_signal_points:
            return CellStatus.PARTIAL
        return CellStatus.UNVISITED

    def record(self, point: Point, result: str) -> None:
        if point not in self.cell.certificate_points:
            raise ValueError(
                "cell evidence may only be recorded at a certificate point"
            )
        if result == "no_signal":
            self.no_signal_points.add(point)
        elif result in {"direction", "near"}:
            self.detected_at = point
        else:
            raise ValueError(f"unsupported result: {result}")


# ---------------------------------------------------------------------------
# Convex feasible-region update and minimum enclosing circle
# ---------------------------------------------------------------------------


def regular_disk_polygon(radius: float, vertices: int = 360) -> list[Point]:
    """Return a regular polygon circumscribing the requested disk."""
    if radius <= 0 or vertices < 3:
        raise ValueError("radius must be positive and vertices must be at least 3")
    vertex_radius = radius / cos(pi / vertices)
    return [
        Point(
            vertex_radius * cos((2 * k + 1) * pi / vertices),
            vertex_radius * sin((2 * k + 1) * pi / vertices),
        )
        for k in range(vertices)
    ]


```

```python
def clip_halfplane(
    polygon: Sequence[Point], normal: Point, bound: float
) -> list[Point]:
    """Clip a convex polygon by normal dot x <= bound."""
    if not polygon:
        return []
    output: list[Point] = []
    previous = polygon[-1]
    prev_val = normal.x * previous.x + normal.y * previous.y - bound
    for current in polygon:
        current_value = normal.x * current.x + normal.y * current.y - bound
        previous_inside = prev_val <= EPS
        current_inside = current_value <= EPS
        if previous_inside != current_inside:
            denominator = prev_val - current_value
            ratio = 0.0 if abs(denominator) <= EPS else prev_val / denominator
            output.append(
                Point(
                    previous.x + ratio * (current.x - previous.x),
                    previous.y + ratio * (current.y - previous.y),
                )
            )
        if current_inside:
            output.append(current)
        previous, prev_val = current, current_value
    return output


def clip_bearing_wedge(
    polygon: Sequence[Point],
    sensor: Point,
    bearing_deg: float,
    error_deg: float = BEARING_ERROR_DEG,
) -> list[Point]:
    """Intersect a convex polygon with a forward bearing wedge."""
    low = (bearing_deg - error_deg) * pi / 180.0
    high = (bearing_deg + error_deg) * pi / 180.0
    u_low = Point(cos(low), sin(low))
    u_high = Point(cos(high), sin(high))
    # cross(u_low, x-sensor) >= 0
    first_normal = Point(u_low.y, -u_low.x)
    first_bound = first_normal.x * sensor.x + first_normal.y * sensor.y
    clipped = clip_halfplane(polygon, first_normal, first_bound)
    # cross(u_high, x-sensor) <= 0
    second_normal = Point(-u_high.y, u_high.x)
    second_bound = second_normal.x * sensor.x + second_normal.y * sensor.y
    return clip_halfplane(clipped, second_normal, second_bound)


def clip_reception_disk(
    polygon: Sequence[Point],
    sensor: Point,
    radius: float = MAX_RECEPTION_RANGE,
    sides: int = 180,
) -> list[Point]:
    """Conservatively intersect with an outer polygon around a reception disk."""
    result = list(polygon)
    # Tangent half-planes form a circumscribed polygon, so no feasible point is
    # incorrectly removed. It is the conservative disk representation used by
    # the online feasible-region update.
    for k in range(sides):
        angle = 2 * pi * k / sides
        normal = Point(cos(angle), sin(angle))
        bound = normal.x * sensor.x + normal.y * sensor.y + radius
```

```
        result = clip_halfplane(result, normal, bound)
        if not result:
            break
    return result


def _circle_from_two(a: Point, b: Point) -> tuple[Point, float]:
    center = Point((a.x + b.x) / 2, (a.y + b.y) / 2)
    return center, distance(a, b) / 2


def _circle_from_three(a: Point, b: Point, c: Point) -> tuple[Point, float] | None:
    denominator = 2 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y))
    if abs(denominator) <= EPS:
        return None
    aa = a.x * a.x + a.y * a.y
    bb = b.x * b.x + b.y * b.y
    cc = c.x * c.x + c.y * c.y
    center = Point(
        (aa * (b.y - c.y) + bb * (c.y - a.y) + cc * (a.y - b.y)) / denominator,
        (aa * (c.x - b.x) + bb * (a.x - c.x) + cc * (b.x - a.x)) / denominator,
    )
    return center, distance(center, a)


def minimum_enclosing_circle(points: Sequence[Point]) -> tuple[Point, float]:
    """Compute the exact MEC by exhaustive candidates for a small polygon."""
    if not points:
        raise ValueError("cannot enclose an empty point set")
    candidates: list[tuple[Point, float]] = [(points[0], 0.0)]
    for i, a in enumerate(points):
        for b in points[i + 1 :]:
            candidates.append(_circle_from_two(a, b))
    for i, a in enumerate(points):
        for j in range(i + 1, len(points)):
            for k in range(j + 1, len(points)):
                circle = _circle_from_three(a, points[j], points[k])
                if circle is not None:
                    candidates.append(circle)
    feasible = [
        (center, radius)
        for center, radius in candidates
        if all(distance(center, point) <= radius + 1e-7 for point in points)
    ]
    return min(feasible, key=lambda item: item[1])
```

```
class Q4Triangle25SurroundPlanner:
    """Rolling planner using 25 vertices of a strict triangular certificate."""

    def __init__(
        self,
        channels: Iterable[int] = range(1, 21),
        total_miss_risk: float = 0.0,
        miss_probability_per_probe: float = 0.5,
        adaptive_route: bool = True,
    ) -> None:
        ch_list = list(channels)
        if total_miss_risk < 0.0 or total_miss_risk >= 1.0:
            raise ValueError("total_miss_risk must be in [0, 1)")
        if not 0.0 < miss_probability_per_probe < 1.0:
            raise ValueError("miss_probability_per_probe must be in (0, 1)")
        self.certificate_cells = build_triangle_cells()
        certificate_points = {
            point
```

```
            for cell in self.certificate_cells
            for point in cell.certificate_points
        }
        if set(STRICT_SEARCH_ROUTE) != certificate_points:
            raise RuntimeError(
                "strict search route does not match certificate point set"
            )
        self.search_route = STRICT_SEARCH_ROUTE
        self.adaptive_route = adaptive_route
        self.search_history: list[Point] = []
        self.channels = {
            channel: ChannelState(
                channel=channel,
                cells={
                    cell.cell_id: CellEvidence(cell) for cell in self.certificate_cells
                },
                polygon=regular_disk_polygon(ARENA_RADIUS),
            )
            for channel in ch_list
        }
        self.total_miss_risk = total_miss_risk
        self.miss_probability_per_probe = miss_probability_per_probe
        self.per_channel_miss_risk = (
            total_miss_risk / len(ch_list)
            if ch_list and total_miss_risk
            else 0.0
        )
        self.probabilistic_negative_limit = (
            ceil(log(self.per_channel_miss_risk) / log(miss_probability_per_probe))
            if self.per_channel_miss_risk
            else None
        )
        self.risk_model = (
            "strict deterministic surround certificate"
            if self.probabilistic_negative_limit is None
            else (
                "model-based: distinct negative search probes are treated as independent; "
                f"P(miss per probe)<={miss_probability_per_probe:g}; union bound over "
                f"{len(ch_list)} channels"
            )
        )

    def pending_stops(self) -> dict[Point, set[int]]:
        """Return strict-search points still required by each unknown channel."""
        stops: dict[Point, set[int]] = {}
        for channel, state in self.channels.items():
            if state.status != ChannelStatus.UNKNOWN:
                continue
            for evidence in state.cells.values():
                if evidence.status in {CellStatus.EXCLUDED, CellStatus.DETECTED}:
                    continue
                for point in evidence.cell.certificate_points:
                    if point not in evidence.no_signal_points:
                        stops.setdefault(point, set()).add(channel)
        return stops

    def next_stop(
        self, current: Point, current_channel: int | None = None
    ) -> Stop | None:
        """Choose the next stop on an exact-length Hamilton route.

        ``current`` is retained in the public interface because the controller
        and route experiments pass the robot position explicitly. The strict
        continuation itself is determined by ``search_history``.
```

```
        """
        candidates = self.pending_stops()
        if not candidates:
            return None
        if not self.adaptive_route:
            point = next(
                (point for point in self.search_route if point in candidates), None
            )
        else:
            point = self._adaptive_hamilton_stop(candidates)
        if point is None:
            raise RuntimeError("pending certificate point is missing from search route")
        self.search_history.append(point)
        channels = candidates[point]
        ordered = sorted(channels, key=lambda c: c != current_channel)
        return Stop(point, tuple(ordered), len(channels))

    def _lazy_information_score(self, point: Point) -> tuple[float, int]:
        values: list[float] = []
        for state in self.channels.values():
            if (
                state.status != ChannelStatus.LOCALIZING
                or point in state.measured_points
            ):
                continue
            center_dist = distance(point, state.mec_center)
            if center_dist - state.mec_radius > MAX_RECEPTION_RANGE + EPS:
                continue
            pred_line = (
                degrees(
                    atan2(state.mec_center.y - point.y, state.mec_center.x - point.x)
                )
                % 180.0
            )
            separation = max(
                (
                    min(
                        abs((pred_line - old) % 180.0),
                        180.0 - abs((pred_line - old) % 180.0),
                    )
                    for old in state.bearing_lines_deg
                ),
                default=90.0,
            )
            proximity = 1.0 / (1.0 + center_dist / GUARANTEED_RANGE)
            values.append(2.0 * separation / 90.0 + proximity)
        values.sort(reverse=True)
        useful = values[:4]
        return sum(useful), len(useful)

    @staticmethod
    def _has_hamilton_completion(start: Point, remaining: frozenset[Point]) -> bool:
        """Check that 1000 m mesh edges can still visit every remaining point."""
        memo: set[tuple[Point, frozenset[Point]]] = set()

        def search(node: Point, left: frozenset[Point]) -> bool:
            if not left:
                return True
            key = (node, left)
            if key in memo:
                return False
            options = [
                point
                for point in left
```

```
                if distance(node, point) <= GUARANTEED_RANGE + MESH_EDGE_TOLERANCE
            ]
            options.sort(
                key=lambda point: sum(
                    distance(point, other) <= GUARANTEED_RANGE + MESH_EDGE_TOLERANCE
                    for other in left
                    if other != point
                )
            )
            for point in options:
                if search(point, left - {point}):
                    return True
            memo.add(key)
            return False

        return search(start, remaining)

    def _adaptive_hamilton_stop(self, pending: dict[Point, set[int]]) -> Point | None:
        visited = set(self.search_history)
        remaining = frozenset(set(TRIANGLE25_POINTS) - visited)
        if not remaining:
            return None
        if not self.search_history:
            options = sorted(
                remaining,
                key=lambda point: (distance(Point(0.0, 0.0), point), point.x, point.y),
            )
            minimum = distance(Point(0.0, 0.0), options[0])
            options = [
                point
                for point in options
                if distance(Point(0.0, 0.0), point) <= minimum + EPS
            ]
        else:
            last = self.search_history[-1]
            options = [
                point
                for point in remaining
                if distance(last, point) <= GUARANTEED_RANGE + MESH_EDGE_TOLERANCE
            ]
        feasible = [
            point
            for point in options
            if self._has_hamilton_completion(point, remaining - {point})
        ]
        if not feasible:
            return None

        def score(point: Point) -> tuple[float, int, int, int, float, float]:
            information, useful = self._lazy_information_score(point)
            route_pref = -self.search_route.index(point)
            return (
                information,
                useful,
                len(pending.get(point, ())),
                route_pref,
                -point.x,
                -point.y,
            )

        return max(feasible, key=score)

    def record(
        self,
```

```
        channel: int,
        point: Point,
        result: str,
        bearing_deg: float | None = None,
    ) -> None:
        state = self.channels[channel]
        state.record(point, result, bearing_deg)
        if (
            result == "no_signal"
            and state.status == ChannelStatus.UNKNOWN
            and self.probabilistic_negative_limit is not None
        ):
            neg_n = len(state.measured_points)
            modeled_risk = self.miss_probability_per_probe**neg_n
            state.modeled_miss_risk = modeled_risk
            if neg_n >= self.probabilistic_negative_limit:
                state.mark_probabilistically_absent(modeled_risk)

    def localization_probe(self, channel: int) -> Point:
        """Return the next orbit point around the current feasible region.

        A compact four-direction pattern is used after the MEC has contracted;
        otherwise a wider exploratory orbit is replanned after every response.
        The persistent phase prevents repeated near-parallel bearings when the
        MEC center moves.
        """
        state = self.channels[channel]
        if state.status != ChannelStatus.LOCALIZING:
            raise ValueError("localization probes require a localizing channel")
        radius = state.mec_radius
        if radius <= GUARANTEED_RANGE / (1.0 + sqrt(2.0)):
            orbit_r = max(30.0, sqrt(2.0) * radius + 1.0)
            angles = (0, 90, 180, 270)
        else:
            # A first bearing often leaves a long wedge. A tiny orbit around
            # its MEC can remain entirely on the emitter's dark side. Use a
            # baseline comparable to the current uncertainty so the path
            # actually crosses to both sides of the source hypothesis.
            orbit_r = min(900.0, max(500.0, radius))
            angles = (0, 180, 90, 270, 45, 225, 135, 315, 30, 210, 120, 300)
        candidates = [
            Point(
                state.mec_center.x + orbit_r * cos(angle * pi / 180.0),
                state.mec_center.y + orbit_r * sin(angle * pi / 180.0),
            )
            for angle in angles
        ]
        # Do not restart at the first angle after every MEC update. The center
        # moves after a positive bearing, so coordinate-based "unused" checks
        # alone would repeatedly select the same side and create near-parallel
        # bearings. Persist the angular phase per channel instead.
        index = state.localization_probe_index
        state.localization_probe_index += 1
        if index < len(candidates):
            return candidates[index]
        # Continue with a low-discrepancy angular sequence after one full orbit.
        phase = index * GOLDEN_ANGLE_DEG
        return Point(
            state.mec_center.x + orbit_r * cos(phase * pi / 180.0),
            state.mec_center.y + orbit_r * sin(phase * pi / 180.0),
        )

    @property
    def complete(self) -> bool:
```

```
        return all(
            state.status in {ChannelStatus.ABSENT, ChannelStatus.CLEARED}
            for state in self.channels.values()
        )

    @property
    def search_route_strategy(self) -> str:
        """Stable identifier used by result summaries and diagnostic logs."""
        return (
            "adaptive_equal_length_hamilton"
            if self.adaptive_route
            else "fixed_optimal_hamilton"
        )
```

```
CANDIDATE_ID = "ring-o12-i12-v000"
INNER_RADIUS_M = 925.0
RING_SIZE = 12
OUTER_RADIUS_M = ARENA_RADIUS / cos(pi / RING_SIZE)
RING25_POINTS = (
    Point(0.0, 0.0),
    *(
        Point(
            INNER_RADIUS_M * cos(pi / RING_SIZE + 2 * pi * index / RING_SIZE),
            INNER_RADIUS_M * sin(pi / RING_SIZE + 2 * pi * index / RING_SIZE),
        )
        for index in range(RING_SIZE)
    ),
    *(
        Point(
            OUTER_RADIUS_M * cos(2 * pi * index / RING_SIZE),
            OUTER_RADIUS_M * sin(2 * pi * index / RING_SIZE),
        )
        for index in range(RING_SIZE)
    ),
)

# Delaunay faces of the fixed concentric construction. Their edge lengths and
# complete polygon tiling are verified at import time below. No SciPy is needed
# for an official run.
RING25_FACES = (
    (10, 11, 0),
    (6, 18, 19),
    (18, 6, 5),
    (5, 6, 0),
    (1, 2, 0),
    (4, 5, 0),
    (20, 21, 8),
    (8, 9, 0),
    (9, 10, 0),
    (21, 9, 8),
    (9, 21, 22),
    (10, 9, 22),
    (23, 10, 22),
    (23, 24, 11),
    (10, 23, 11),
    (11, 12, 0),
    (12, 1, 0),
    (24, 12, 11),
    (12, 24, 13),
    (1, 12, 13),
    (14, 1, 13),
    (14, 15, 2),
    (1, 14, 2),
    (17, 18, 5),
```

```
    (4, 17, 5),
    (17, 4, 16),
    (15, 3, 2),
    (4, 3, 16),
    (3, 15, 16),
    (2, 3, 0),
    (3, 4, 0),
    (7, 6, 19),
    (7, 20, 8),
    (20, 7, 19),
    (7, 8, 0),
    (6, 7, 0),
)

# Fixed open route from the Pareto experiment. It starts with a measurement at
# the initial position (origin), then visits every other certificate vertex.
RING25_ROUTE_INDICES = (
    0,
    10,
    11,
    12,
    1,
    2,
    3,
    4,
    5,
    6,
    7,
    8,
    9,
    21,
    22,
    23,
    24,
    13,
    14,
    15,
    16,
    17,
    18,
    19,
    20,
)
RING25_ROUTE = tuple(RING25_POINTS[index] for index in RING25_ROUTE_INDICES)


def _triangle_area(cell: TriangleCell) -> float:
    a, b, c = cell.vertices
    return abs((b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x)) / 2


def build_ring25_cells() -> list[TriangleCell]:
    """Build and verify the fixed 36-triangle strict coverage certificate."""
    cells = [
        TriangleCell(f"r{index:02d}", tuple(RING25_POINTS[i] for i in face))
        for index, face in enumerate(RING25_FACES)
    ]
    if len(RING25_POINTS) != 25 or len(cells) != 36:
        raise RuntimeError("ring25 point or triangle count changed")
    if set(RING25_ROUTE) != set(RING25_POINTS) or len(set(RING25_ROUTE)) != 25:
        raise RuntimeError("ring25 route does not visit every certificate point once")
    if not all(cell.certifiable(GUARANTEED_RANGE) for cell in cells):
        raise RuntimeError("ring25 triangle exceeds guaranteed reception range")

```

```
    # A planar triangulation with n=25 and h=12 has 2n-h-2=36 faces. Check
    # that all non-boundary edges are shared twice, all outer edges once, and
    # the summed face area equals the circumscribed regular polygon area.
    edge_counts: dict[tuple[int, int], int] = {}
    for face in RING25_FACES:
        for a, b in ((face[0], face[1]), (face[1], face[2]), (face[2], face[0])):
            edge = (min(a, b), max(a, b))
            edge_counts[edge] = edge_counts.get(edge, 0) + 1
    outer_edges = {
        (
            min(13 + index, 13 + (index + 1) % RING_SIZE),
            max(13 + index, 13 + (index + 1) % RING_SIZE),
        )
        for index in range(RING_SIZE)
    }
    if (
        any(
            count != (1 if edge in outer_edges else 2)
            for edge, count in edge_counts.items()
        )
        or set(edge for edge, count in edge_counts.items() if count == 1) != outer_edges
    ):
        raise RuntimeError("ring25 triangles do not form the expected mesh")
    polygon_area = RING_SIZE * OUTER_RADIUS_M**2 * sin(2 * pi / RING_SIZE) / 2
    if abs(sum(_triangle_area(cell) for cell in cells) - polygon_area) > 1e-5:
        raise RuntimeError("ring25 triangles do not tile the outer polygon")
    if abs(OUTER_RADIUS_M * cos(pi / RING_SIZE) - ARENA_RADIUS) > EPS:
        raise RuntimeError("ring25 outer ring does not circumscribe the arena")
    return cells


RING25_CELLS = build_ring25_cells()
RING25_ROUTE_LENGTH_M = sum(
    distance(a, b) for a, b in zip((Point(0.0, 0.0), *RING25_ROUTE), RING25_ROUTE)
)


class Q4Ring25Planner(Q4Triangle25SurroundPlanner):
    """The existing Q4 state machine with the concentric strict certificate."""

    def __init__(
        self,
        channels: Iterable[int] = range(1, 21),
        total_miss_risk: float = 0.0,
        miss_probability_per_probe: float = 0.5,
    ) -> None:
        ch_list = list(channels)
        super().__init__(
            channels=ch_list,
            total_miss_risk=total_miss_risk,
            miss_probability_per_probe=miss_probability_per_probe,
            adaptive_route=False,
        )
        self.certificate_cells = RING25_CELLS
        self.search_route = RING25_ROUTE
        self.channels = {
            channel: ChannelState(
                channel=channel,
                cells={
                    cell.cell_id: CellEvidence(cell) for cell in self.certificate_cells
                },
                polygon=regular_disk_polygon(ARENA_RADIUS),
            )
            for channel in ch_list
```

```
        }

    @property
    def search_route_strategy(self) -> str:
        return "fixed_concentric25_2opt"
```